HIGH-LEVEL DESIGN · CONCEPTS HANDBOOK

HLD Notes Foundational to Advanced

Every core building block of system design: networking, scaling, caching, databases, APIs, messaging, security, and the distributed-systems theory behind them.

FOUNDATIONAL · fresher level **ADVANCED · senior / skip if short on time**

Concept module. Worked HLD walkthroughs (WhatsApp, Uber, …) are a separate module.

Contents

0 How to use this book 3

PART 1 · FOUNDATIONAL

1 How a request travels: client, server, IP, DNS FOUNDATIONAL 6
2 Network protocols: TCP, UDP, HTTP versions, HTTPS FOUNDATIONAL 8
3 Scaling, availability and load balancing FOUNDATIONAL 11
4 Caching and CDN FOUNDATIONAL 14
5 Databases: choosing, indexing and storing data FOUNDATIONAL 16
6 API design with REST FOUNDATIONAL 20
7 GraphQL, gRPC and choosing an API style FOUNDATIONAL 23
8 Real-time communication and webhooks FOUNDATIONAL 25
9 Sync vs async, message queues and events FOUNDATIONAL 28
10 Monolith, microservices and the API gateway FOUNDATIONAL 31
11 Authentication, authorization and API security FOUNDATIONAL 33
12 CAP theorem and consistency FOUNDATIONAL 38
13 Back-of-the-envelope estimation FOUNDATIONAL 40

PART 2 · ADVANCED

14 Database sharding and consistent hashing ADVANCED 42
15 Replication, consensus and distributed transactions ADVANCED 45
16 Messaging in depth: RabbitMQ and Kafka ADVANCED 48
17 Storage engines and indexing internals ADVANCED 51
18 Specialised stores: DynamoDB, Elasticsearch, time series, probabilistic structures ADVANCED 53
19 Redis in depth ADVANCED 55
20 Real-time at scale: WebSockets and WebRTC ADVANCED 57
21 Advanced security and identity ADVANCED 59
22 Infrastructure and theory odds and ends ADVANCED 61

PART 3 · SUPPLEMENT

23 Supplement: reliability, caching failures, IDs and concurrency FOUNDATIONAL 63
24 Supplement: observability, deployment and data pipelines ADVANCED 66

PART 4 · INTERVIEW PLAYBOOK

25 Which component for which problem 68

0 How to use this book

Every chapter is tagged FOUNDATIONAL or ADVANCED. Foundational chapters are what a fresher needs to hold a system-design conversation. Advanced chapters (sharding internals, consensus, Kafka/RabbitMQ internals, storage engines, WebRTC…) matter for senior rounds; **skip the whole of Part 2 if time is short**. Foundational chapters end with a purple *"Going deeper"* pointer to the matching advanced chapter.

Reading plans

Time you have	What to read
20 minutes	Chapter 25 (component picker + interview checklist) and the *Remember* / *Trap* boxes of Part 1.
Fresher prep	All of Part 1 (Chapters 1–13), then Chapters 23 and 25.
Part 3 (Chapters 23–24) is **added supplement material** (not from your lectures) covering common interview topics the lectures skipped: reliability patterns, delivery semantics, cache failures, SLAs, DR, locking, IDs, observability, deployment, data pipelines.	
Senior prep	Everything. Part 2 chapters are independent; read them in any order.

Reading the boxes

REMEMBER
The core idea worth memorising.

INTERVIEW TRAP
Common wrong answers and follow-up questions.

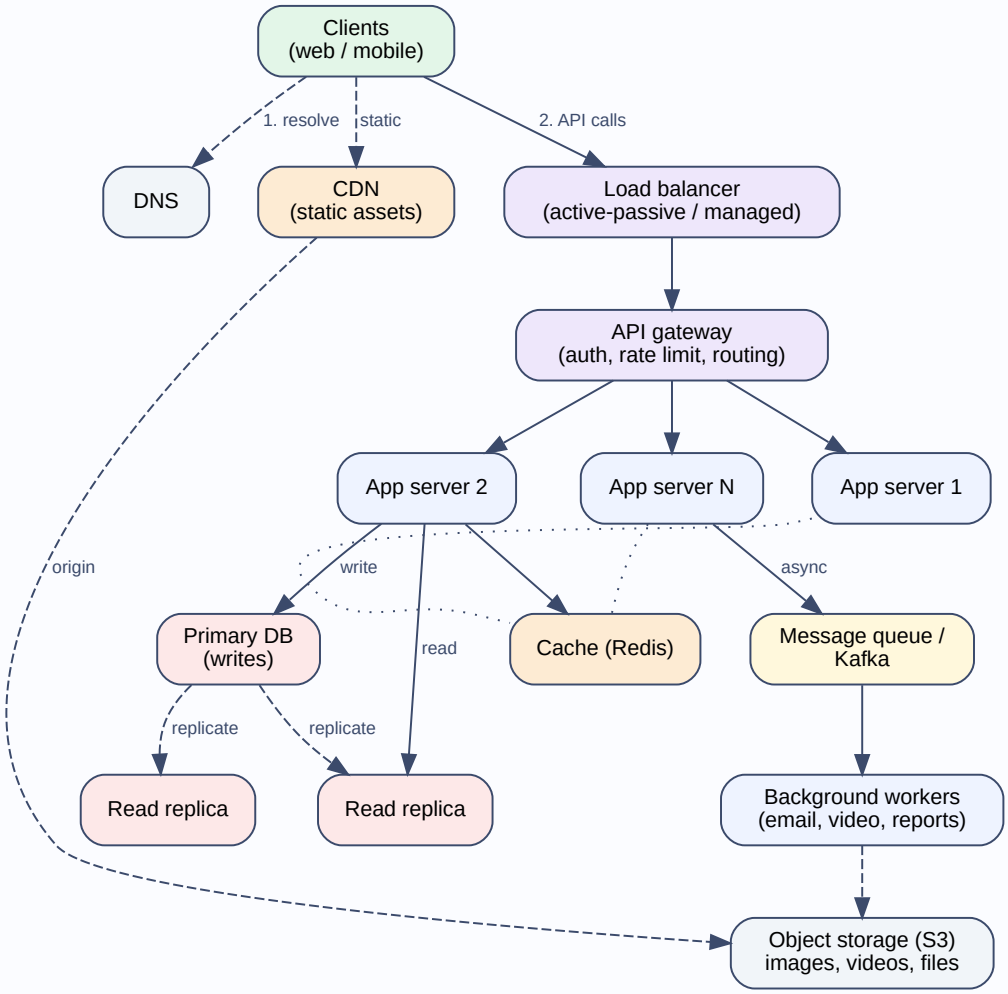
SAY THIS
A ready-made sentence or decision rule for the interview.

REAL-WORLD USE
Where the idea shows up in production.

Going deeper → pointer from a foundational topic to its advanced chapter.

The mental model in one picture

Almost every HLD answer is a variation of this reference architecture. Each box is a chapter in this book.



Reference architecture. Clients resolve via DNS (Ch 1), pass through a load balancer (Ch 3) and API gateway (Ch 10) to stateless app servers, which use a cache (Ch 4), a replicated database (Ch 5, 15), queues for async work (Ch 9, 16) and object storage for files (Ch 5).

Sources

These notes consolidate the following playlists and videos. Where a chapter comes from the numbered lecture series, it carries a `▶ L#` tag matching the lecture number in the original notes (the lecture series is the first playlist below).

#	Source
1	youtube.com/playlist?list=PLlvkTvW25FtjlM61SlOcLS05SfwoYgMit
2	youtube.com/playlist?list=PLMCXHnjXnTnvo6alSjVkgxV-VH6EPyvoX
3	youtube.com/playlist?list=PLA3GkZPtsafZdyC5iucNM_uhqGJ5yFNUM
4	youtube.com/watch?v=BHCSL_ZifI0&list=PL5q3E8eRUieUHnsz0rh0W6AzwdVJBwEK6&index=5

PART 1

Part 1 · Foundational

Thirteen chapters at fresher level: how requests travel, how systems scale, where data lives, how services talk and stay secure, and how to estimate.

1 How a request travels: client, server, IP, DNS

FOUNDATIONAL

▶ Source: L2, crash course

Client and server

- **Client:** anything that initiates a request: browser, mobile app, or another upstream service.
- **Server:** a machine that is always on (self-hosted or cloud, e.g. AWS EC2), runs the business logic, and responds with data.

	Web app	Mobile app
Response	Heavy: HTML, CSS, JavaScript (page + presentation)	Light: **JSON** payload only (easy to parse, small)
Why	The browser renders the UI from the response	The app already has its own UI; it only needs data via HTTP APIs

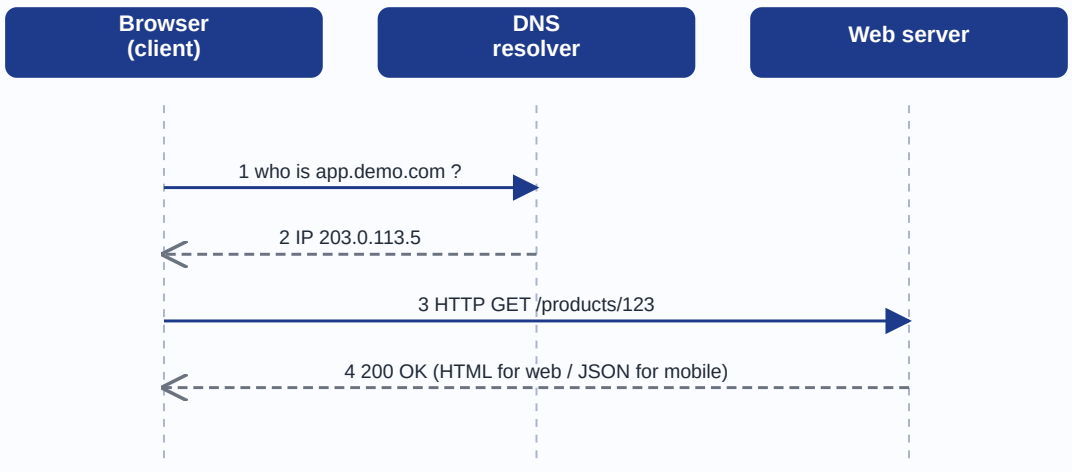
IP addresses

	IPv4	IPv6
Size	32 bits = 4 octets (e.g. `203.0.113.5`)	128 bits = 8 hex groups
Capacity	~4.3 billion addresses; running out	~3.4×10^{38}; exhaustion solved

- **Public IP:** globally unique, assigned by your ISP, reachable from the internet.
- **Private IP:** assigned by your router to local devices (laptop, phone, TV); invisible to the internet.
- **NAT (Network Address Translation):** the router maps many private IPs to one public IP for outgoing traffic and routes replies back.

DNS: the internet's phonebook

Humans remember `amazon.com`; machines route by IP. DNS translates the domain into an IP address.



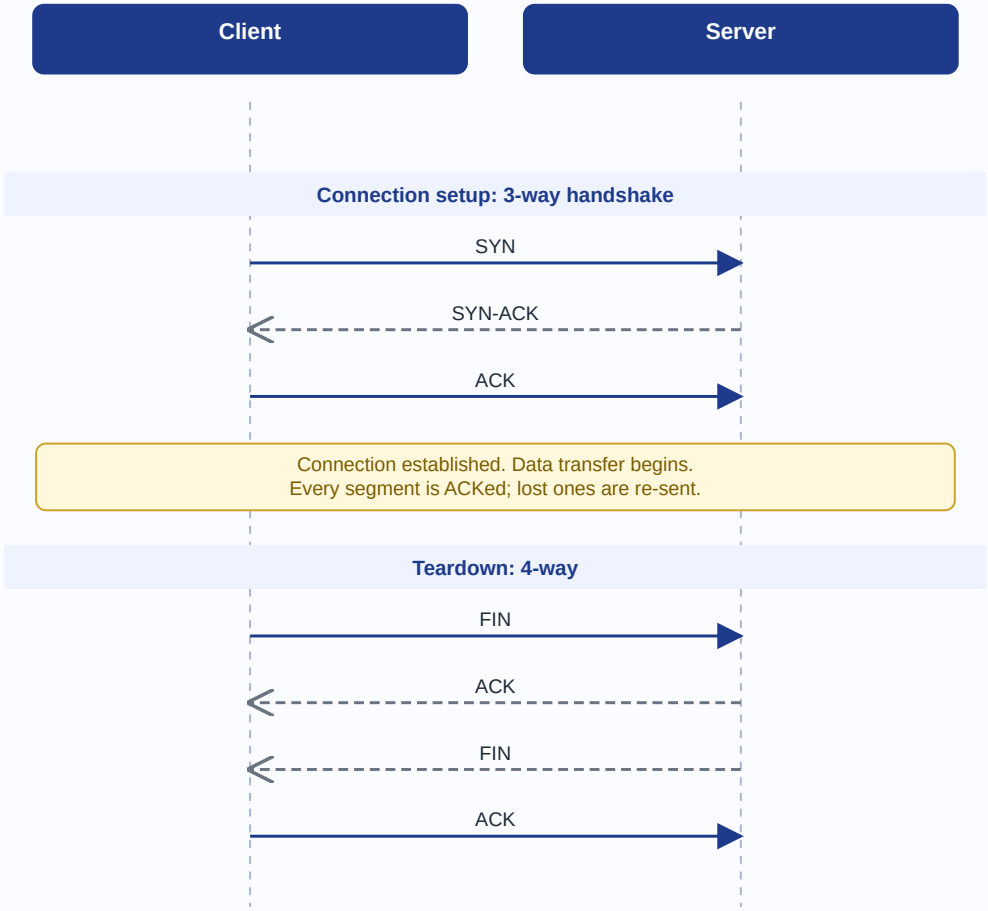
Full flow when a user opens a site: DNS lookup first, then the HTTP request to the returned IP.

> **REMEMBER**
> Two layers are enough for interviews: the **application layer** (HTTP, HTTPS, WebSockets, WebRTC) and the **transport layer** (TCP, UDP). Next chapter covers them.

2 Network protocols: TCP, UDP, HTTP versions, HTTPS FOUNDATIONAL

▶ Source: L2, L9, L29

TCP vs UDP



TCP handshake (SYN, SYN-ACK, ACK) and 4-way teardown (FIN/ACK).

	TCP	UDP
Connection	Connection-oriented: 3-way handshake first	Connectionless: just send
Reliability	ACKs, retransmission of lost packets, **ordered** delivery	No guarantee, no retransmit, no order
Extras	Flow control (don't overflow receiver) and congestion control	None; minimal overhead
Pick it for	Web, APIs, databases, payments, email, file transfer	Live video, gaming, VoIP, video calls

INTERVIEW DEFAULT

"I'll use **TCP** wherever data loss is unacceptable. I'll switch to **UDP** only when low latency matters more than occasional lost packets, like live streaming or game state."

HTTP/1.0 → 1.1 → 2 → 3

Version	Transport	Key idea / limitation
1.0	TCP	Text. **New TCP connection per request** (handshake each time). Painfully slow for pages with many assets.
1.1	TCP	Persistent connections (**keep-alive**) reuse one TCP connection. But responses are strictly in order: a slow request A blocks B and C behind it (**application-layer head-of-line blocking**).
2	TCP	**Binary framing**, **multiplexing** (parallel streams on one connection), **HPACK** header compression, server push. Flaw: one lost TCP packet stalls *all* streams (**TCP-level HoL blocking**).
3	QUIC over **UDP**	Independent streams (loss in one doesn't block others), TLS 1.3 built in, **0-RTT** resume, survives Wi-Fi ↔ mobile switching. Used for streaming, gaming, mobile.

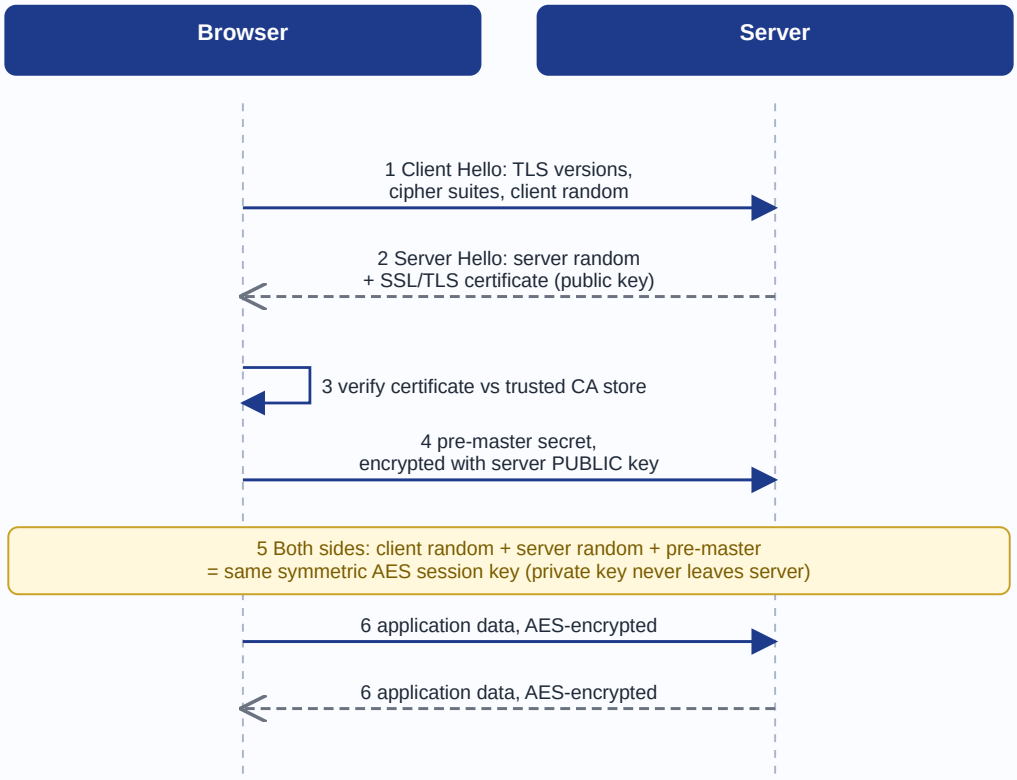
MYTH TO CORRECT

"REST = HTTP/1.1 and gRPC = HTTP/2" is wrong. REST is an architectural style and can run on HTTP/1.1, 2 or 3; most REST stacks just don't exploit HTTP/2. gRPC was designed for HTTP/2 from day one.

HTTPS = HTTP + TLS

HTTP sends plain text (passwords, cookies readable by anyone on the path). **HTTPS** (port **443**) encrypts it and proves server identity. SSL is the legacy name; TLS is the modern protocol.

- **Why not just public-key encryption?** (1) A hacker can intercept the key request and substitute their own public key (**man-in-the-middle**). (2) Asymmetric crypto is too slow for bulk data.
- **Certificate Authority (CA)** (e.g. Let's Encrypt) verifies domain ownership and signs a certificate containing the domain, the server's **public key**, validity dates and the CA's **digital signature**. The private key is never in the certificate.
- **Signature check:** the CA hashed the cert data and encrypted the hash with its private key. The browser hashes the cert itself (H1), decrypts the signature with the CA's public key from its built-in trust store (H2); **H1 = H2** means authentic and untampered.
- **Hybrid design:** use asymmetric crypto only to agree on a secret, then **symmetric AES** for all data (fast).



TLS handshake: certificate check, pre-master secret, then a shared AES session key.

Going deeper → mTLS (both sides present certificates) and PKCE are in Chapter 21. gRPC internals in Chapter 21.

3 Scaling, availability and load balancing FOUNDATIONAL

▶ Source: L3, crash course

Why systems must scale

A single server handles ~50–100 users comfortably; thousands or millions exhaust CPU, memory or bandwidth. **Scaling** means adding capacity when traffic grows (scale up) and removing it when traffic drops (scale down). First separate the **web/app tier** from the **data tier** so each scales independently.

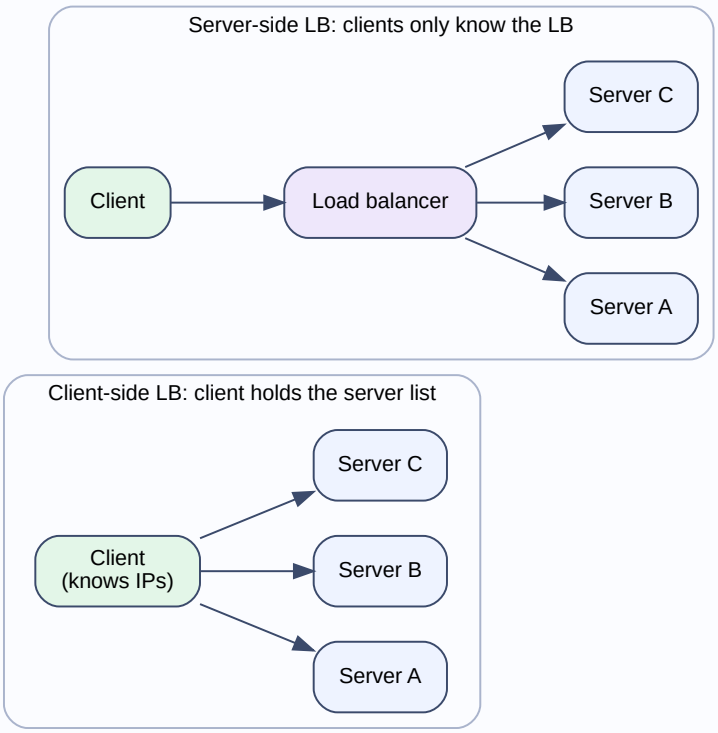
> **SINGLE POINT OF FAILURE (SPOF)**
> Any component whose failure takes the whole system down. A single server, a single database and a single load balancer are all SPOFs. High availability = **redundancy + monitoring + self-healing** (auto-replace failed instances).

	Vertical (scale up)	Horizontal (scale out)
What	Bigger machine: more CPU/RAM	More machines in a pool
Analogy	Motorcycle → car → bus	Run many buses in parallel
Pros	Simple, no code changes	No SPOF, elastic, industry standard
Cons	Hardware ceiling, expensive, downtime to upgrade, **still a SPOF**	Complexity: needs load balancing and state management
Use when	Small app, predictable load, can tolerate downtime	Production systems and anything large

> **KEY ENABLER: STATELESS SERVERS**
> To scale out, app servers should not keep user state in local memory; any server must be able to handle any request. State (sessions, carts) goes to a shared store like Redis or the database.

Load balancer (LB)

A component that spreads incoming requests across healthy servers. It continuously runs **health checks** and stops routing to dead nodes.



Client-side LB (client picks a server; low latency but stale server lists) vs server-side LB (extra hop but hides topology).

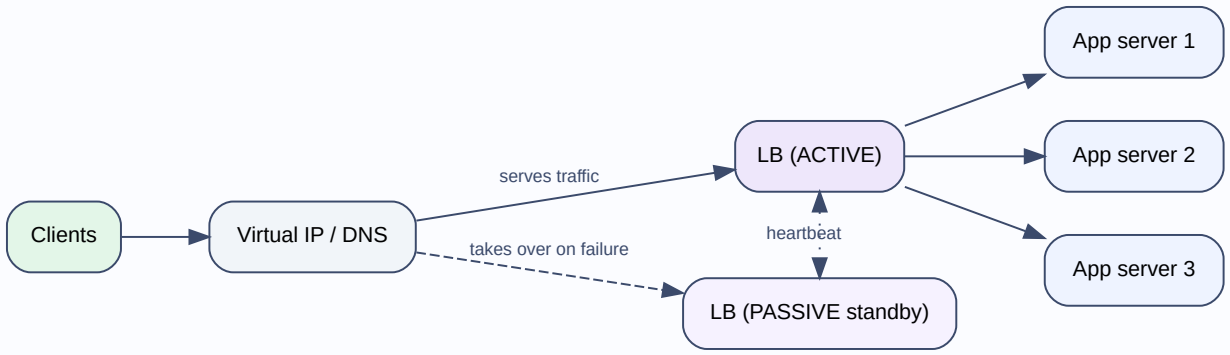
Algorithms: pick by context

Algorithm	How it works	Best for
Round robin	Servers in circular order	Identical servers, similar requests
Weighted round robin	Bigger servers get proportionally more	Mixed hardware (64 GB vs 16 GB)
Least connections	Send to the server with fewest active connections	Sessions of very different lengths
Least response time	Fastest-responding node wins	Latency-sensitive services
IP hash	Hash of client IP → same server	Sticky sessions / persistence
Random	Pick any server	Simple, large homogeneous pools
Geographical	Nearest data centre	Global users, latency
Consistent hashing	Hash ring; adding/removing a server moves only ~1/N of clients	Caches and stateful routing (see Ch 14)

Layer 4 vs Layer 7

	L4 (transport)	L7 (application)
Routes on	IP + port only; payload not inspected	URL path, HTTP method, headers, cookies
Strength	Fast; keeps persistent TCP connections	Smart routing (`/orders` → order service)
Use for	WebSockets, raw TCP, max throughput	HTTP/HTTPS web traffic, microservice routing

Making the LB itself highly available



Active-passive pair behind a virtual IP / DNS. Active-active runs both at once. Cloud LBs (AWS ALB/NLB, Azure LB) do this for you.

Types: software (NGINX, HAProxy), hardware (F5, Citrix), cloud-managed (AWS ELB, Azure, GCP). Same rule for the **API gateway** (Ch 10) and database: redundancy, monitoring, self-healing.

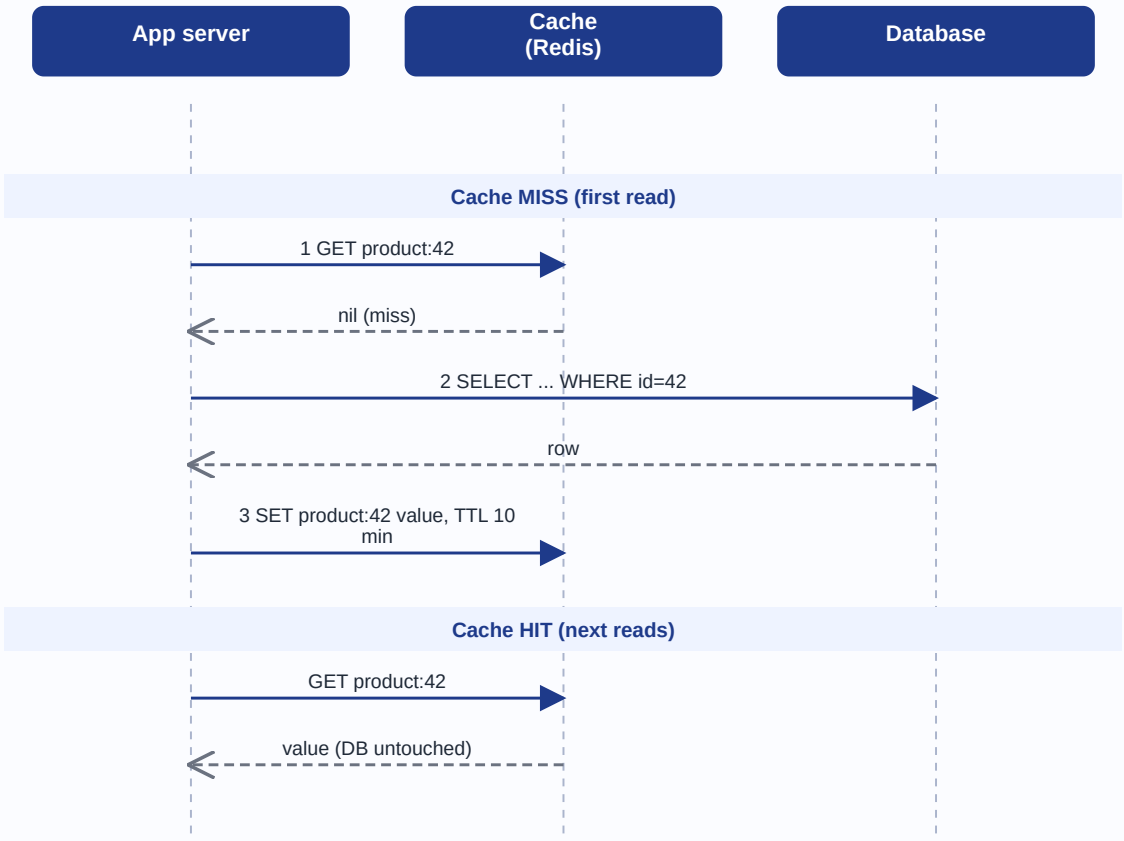
> **Going deeper →** consistent hashing and sharding: Ch 14. Replication of the data tier: Ch 15.

4 Caching and CDN FOUNDATIONAL

▶ Source: L4, L23 (Redis), crash course

Why cache

- The same hot data is requested repeatedly, or a query is expensive (multi-table JOINs). Disk-based databases get overloaded.
- A cache keeps data in **RAM** (far faster than disk): lower latency and less DB load.
- Caches are **volatile** (lost on restart), **small** (RAM is expensive) and **rebuildable**: the database stays the single source of truth.
- Best for data that is **read often and changes rarely**: restaurant menus, product details, profile pictures. Tools: **Redis**, Memcached.



Cache-aside (lazy loading): hit returns from RAM; miss reads the DB and populates the cache with a TTL.

Keeping the cache correct: invalidation and write strategies

Strategy	Mechanics	Trade-off	Use for
TTL	Entries auto-expire after N minutes	Trivial, but data may be stale until expiry	Catalogues, avatars
Write-through	Write to cache **and** DB together	Strong consistency; slower writes (two stores)	Balances, payments
Write-around	Write to DB only, skip cache	No cache pollution from write-heavy, rarely-read data; first read after write is a miss	Logs, bulk data

Write-back (behind)	Write to cache, persist to DB asynchronously	Fastest writes; **data loss** if cache dies first	Counters, tolerant logging

Eviction when the cache is full

Policy	Evicts
FIFO	Oldest inserted item, regardless of use
LRU	Item not accessed for the longest time (usual default)
LFU	Item with the lowest access count

Coding-round favourite: implement LRU/LFU with a hash map + doubly linked list.

CDN: caching at the edge

A **Content Delivery Network** is a global fleet of edge servers. A request hits the nearest edge; on a miss the CDN fetches from the **origin**, caches it locally and replies. It stores static assets (images, video, CSS, JS) and frequently read API responses. Providers: Cloudflare, Akamai, AWS CloudFront. **Anycast** lets one IP route each user to the nearest edge.

Client-side caching

Browser/HTTP caching (via `Cache-Control` headers), local storage and in-app memory avoid network calls, enable offline use and cut API latency.

> **INTERVIEW TRAP**
> "What about stale data?" Always answer with **TTL + an invalidation/write strategy** and state the consistency you accept. Also mention that caching trades consistency for speed (see CAP, Ch 12).

> **Going deeper →** Redis data structures, cluster, hot keys, distributed locks and rate limiting: Ch 19. Bloom filters in front of caches: Ch 18.

5 Databases: choosing, indexing and storing data

FOUNDATIONAL

▶ Source: L20, L27, crash course, DB indexing video

SQL (relational)

Data in **tables** (rows = records, columns = attributes). **Primary key** identifies a row; **foreign keys** link tables; **joins** combine them. Examples: PostgreSQL, MySQL, Oracle.

> **ACID**
> **A**tomicity: all or nothing (a bank transfer fully completes or fully rolls back). **C**onsistency: moves from one valid state to another, honouring constraints. **I**solation: concurrent transactions don't interfere. **D**urability: once committed, survives crashes.

NoSQL: four families

Type	Examples	Model and strength
Document	MongoDB, Firestore	JSON-like docs with nested data; **flexible schema** (shoes have sizes, speakers have battery specs); nesting avoids joins
Key-value	Redis, Memcached, DynamoDB	Lookup by key only; extreme speed; caching and sessions
Wide-column	Cassandra, HBase	Huge **write-heavy** workloads, time series, logs, global scale
Graph	Neo4j, Neptune	Nodes + edges; multi-hop relationships (friends of friends, recommendations)

How to choose

	SQL	NoSQL
Schema	Fixed (or JSONB columns)	Flexible
Consistency	Strong ACID by default	Often eventual (tunable)
Scaling	Vertical, read replicas, sharding, caching, connection pooling	Native horizontal scaling
Best at	Joins, complex queries, money and orders	High write rate, flexible payloads, specialised reads
Interview role	**Primary source of truth**	Specialised workloads and caching

> **THE GOLDEN DEFAULT**
> "Unless the requirements say otherwise, I'll use **PostgreSQL**." It handles structured data and also `JSONB` documents, key-value-style lookups and full-text search. SQL vs NoSQL is no longer a rigid divide: SQL gained JSON, and MongoDB gained ACID transactions.

INTERVIEW TRAP: "SQL CAN'T SCALE"

False. It scales with **read replicas, sharding, caching and connection pooling**. Real systems use **polyglot persistence**: PostgreSQL/MySQL for core data (orders, payments), Redis for cache and sessions, MongoDB for catalogues, Cassandra for logs and activity.

Indexing: why queries get fast

Databases store rows in disk **pages** (~8 KB, ~100 rows). Without an index, a lookup scans pages sequentially (**full table scan**). An index maps keys to pages.

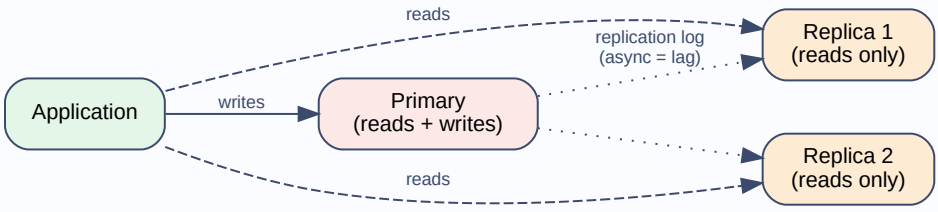
Index	Idea	Use
B-tree	Self-balancing sorted tree; the default in relational DBs	Point lookups, range queries (`age > 50`), sorting
Hash	Hash table, O(1) exact match	Equality only; no ranges/sort. Common in Redis
Inverted	Word → list of documents containing it	Full-text search (Elasticsearch, Postgres FTS)
Geospatial	Geohash, quadtree, R-tree (2D data)	"Nearby" queries (Ch 17)

COST OF INDEXES

Each index speeds reads but slows writes and uses space. Index the columns you filter/sort on, not everything.

Replication basics

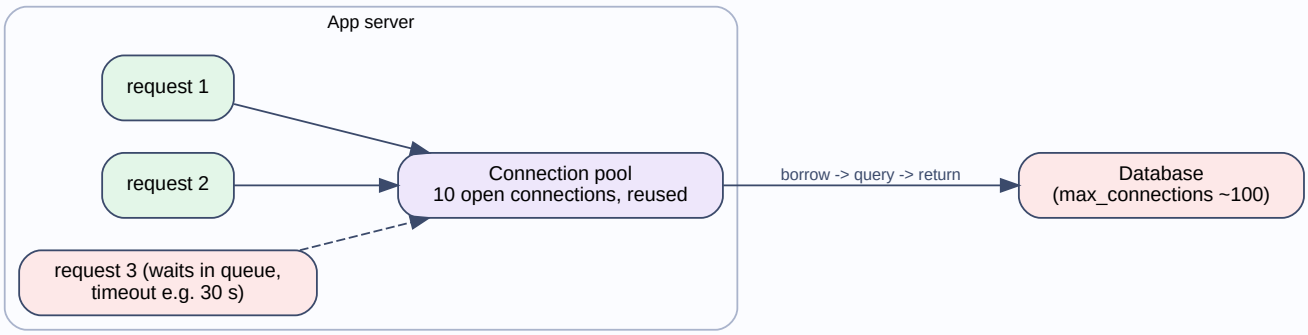
One DB = a SPOF. **Primary-replica (master-slave):** the primary takes writes; replicas copy its log and serve reads. Scales **reads** and gives failover (a replica is promoted if the primary dies). Trade-off: **replication lag** means replicas may serve slightly stale data.



Writes go to the primary; reads can go to replicas; replication is asynchronous.

Connection pooling

Opening a DB connection is expensive (PostgreSQL spawns an OS process per connection) and DBs cap connections (~100 by default; beyond that: `Too many clients`). A **pool** keeps a fixed set of open connections that requests borrow and return; when all are busy, requests wait in a queue until a timeout.



Requests borrow pooled connections; extras queue with a timeout instead of overwhelming the DB.

POOL SIZING MYTH

A bigger pool is **not** faster: too many connections cause CPU context-switching and steal RAM from the DB cache. Rule of thumb: **pool ≈ 2 × CPU cores**, divided across app servers, then load-test.

Object storage: don't put files in your DB

BLOB = Binary Large Object (images, video, PDFs). Databases can store them, but at scale it is an anti-pattern:

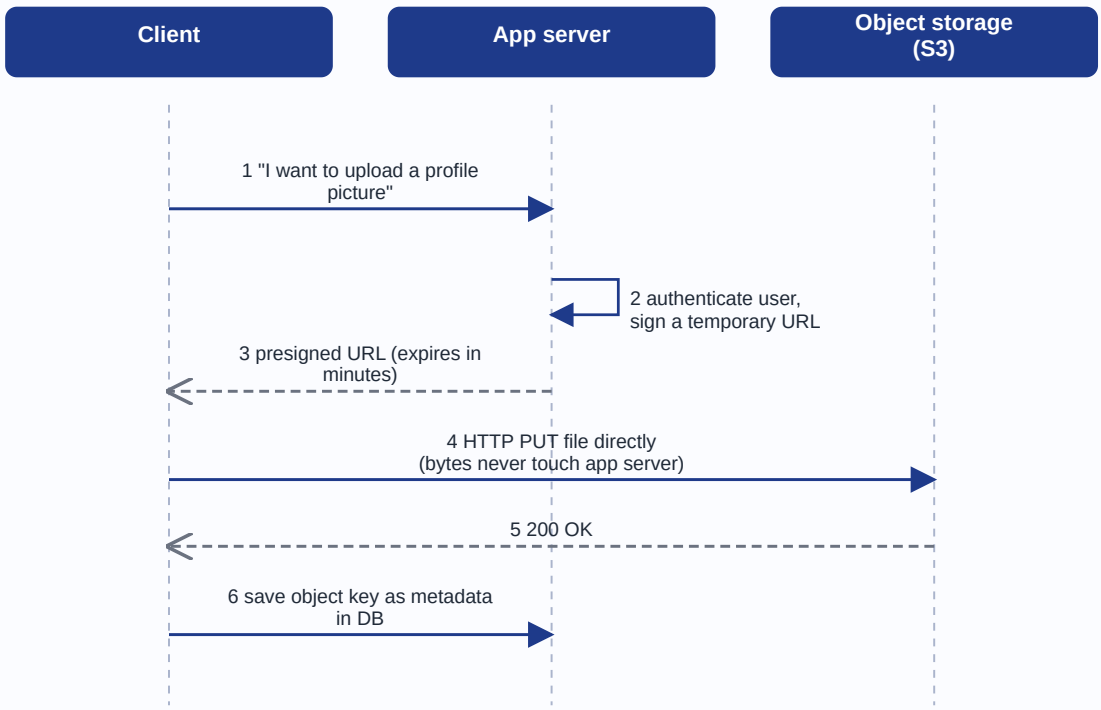
- Large reads evict useful index pages from the DB buffer cache; `SELECT *` drags megabytes over the network.
- Backups, point-in-time recovery and replication become huge and slow; DB storage (fast SSD) is far costlier than object storage.
- Every file would have to proxy through app servers, wasting their CPU, memory and bandwidth.

Correct pattern: store **metadata** (user_id, file name, size, object key/URL) in the DB and the **bytes in object storage** (Amazon S3, GCS, Azure Blob).

Property	Meaning
Objects in buckets	Accessed over HTTP (`GET/PUT/DELETE`); each object = bytes + metadata + unique key
Immutable	No in-place edits; overwrite or version
Flat namespace	`photos/2025/a.png` is one string key; "folders" are a UI prefix illusion
11 nines durability	Erasure coding (K+M blocks) + multi-region replication
Multipart upload	Huge files split into parallel, resumable parts

Production upload: presigned URLs

Never stream a 5 GB video through the app server. Instead the app server authorises the user and hands out a temporary **presigned URL**; the client uploads straight to S3 (and downloads via a CDN).



Presigned-URL upload: the app server stays out of the data path.

6 API design with REST FOUNDATIONAL

▶ Source: L6, L7, L11

An **API** lets a client talk to a service without seeing its internals: the waiter takes your order to the kitchen and brings food back. **REST** (Representational State Transfer) builds APIs on HTTP(S), URLs and verbs, usually with JSON.

REST principles

1. **Client-server separation:** UI and backend evolve independently.
2. **Stateless:** every request carries everything needed; the server keeps no client state between requests (this is why sessions break the REST principle, Ch 11).
3. **Uniform interface:** standard HTTP methods across all resources.
4. **Resource naming: plural nouns** (`/users` , `/orders/456`), never verbs (`/getProducts` , `/deleteUser` are anti-patterns).
5. **Standard formats:** JSON (lightweight, readable) or XML.

Methods = CRUD

Method	Meaning	Example	Idempotent?
`GET`	Read	`GET /users/123`	Yes
`POST`	Create	`POST /users` → `201 Created`	**No**
`PUT`	Replace whole resource	`PUT /users/123`	Yes
`PATCH`	Partial update	`PATCH /users/123` {"active":true}	Usually
`DELETE`	Remove	`DELETE /users/123` (then GET → 404)	Yes

> **IDEMPOTENT**
> Repeating the request leaves the same end state. Critical for retries: a retried `POST /payments` may charge twice, so use an **idempotency key** (client-sent unique ID the server remembers) for payment-like POSTs.

Path vs query parameters

- **Path:** identifies *one* resource: `/users/123` .
- **Query** (after `?` , joined by `&`): **filter** (`?active=true`), **sort** (`?sortBy=name`), **paginate** (`?page=2&limit=10`).

Status codes

Code	Meaning
200 / 201	OK / Created
3xx	Redirection
400	Bad request: invalid syntax or payload
401	Not *authenticated* (missing/invalid credentials). Despite the name "Unauthorized"

403	Authenticated but *not permitted* (e.g. user hitting `/admin/report`)
404	Resource not found
429	Too many requests (rate limited)
4xx / 5xx	Client error / server error

Headers: request/response metadata

Header	Purpose
`Authorization`	Credentials: API key, Basic, `Bearer <token>`
`Content-Type` / `Accept`	Format of the body sent / format the client wants back (`application/json`)
`User-Agent`, `Host`	Client info; target domain
`Cache-Control`	Caching directives (e.g. `no-cache`)
`Set-Cookie`	Server tells the browser to store a cookie (sessions)
`Access-Control-Allow-Origin`	CORS: which cross-domain browsers may call the API
`Content-Length`, `Date`	Body size; send timestamp

Pagination: never return everything

An unpaginated list means high latency, wasted bandwidth (an 8 MB payload when the user sees 1 MB) and overloaded servers. Split results into pages and let the client control size.

	Offset / page-based	Cursor-based
Request	`?page=3&limit=20` or `?offset=40&limit=20`. **offset = (page − 1) × limit**	`?limit=20&cursor=eyJ…` (opaque Base64 bookmark of the *last item seen*: `created_at` + `id`)
SQL	`ORDER BY created_at DESC, id DESC LIMIT 20 OFFSET 40`	`WHERE (created_at, id) < (:ts, :id) ORDER BY … LIMIT 20`
Speed	Deep pages are slow: DB scans and discards N rows (O(N))	Indexed lookup, stays fast at any depth
Under concurrent writes	**Duplicates / skips** (see below)	**Consistent**: relative to an item, not a position
"Jump to page 17"	Yes	No: only next/previous
Complexity	Low	Higher: encode, sign, expire cursors; a cursor is bound to its sort/filter
Best for	Admin tables, small/static data, search results	Feeds, infinite scroll, high-write streams

> **WHY OFFSET BREAKS**
> Page 1 returns IDs 10, 9. A new item 11 is inserted; page 2 (offset 2) now starts at 9, so **9 appears twice**. Delete an item on page 1 instead and rows shift up, so page 2 **skips** a record. Cursor pagination is immune.

Best practices

- **Cap the limit** (e.g. max 100); reject or switch to cursors beyond a safe offset (~10k).
- Return `has_more` by fetching `limit + 1` rows instead of a costly `COUNT(*)`; cache totals in Redis if needed.
- Cursors: opaque, **signed** against tampering, with a short TTL; a changed filter invalidates the cursor (`400`).
- **Keyset (seek)** pagination = same idea with raw keys in the URL (`?after_id=96`). **Time-window** pagination (`?start=…&end=…`) suits logs and time series.

Versioning and when REST fits

Version breaking changes (`/v1/orders` , `/v2/orders`). REST suits standard CRUD web/mobile apps and public APIs (Stripe, Google Maps). It struggles with real-time push (use WebSockets), over/under-fetching (GraphQL) and ultra-fast internal calls (gRPC), covered next.

7 GraphQL, gRPC and choosing an API style

FOUNDATIONAL

▶ Source: L8, L10, L12, L13

GraphQL: the client chooses the fields

REST is a fixed set meal; GraphQL is à la carte. One endpoint (`/graphql`) accepts queries describing exactly the fields wanted, including nested relations, in a **single round trip**.

- **Solves over-fetching** (100 fields returned when the UI needs 2), **under-fetching** (several REST calls for user → orders → items → products) and URL version churn (deprecate individual fields instead).
- **Strongly typed schema (SDL):** a contract of types, queries and mutations; requests are validated against it (a misspelt field fails immediately); **introspection** lets clients discover it.

Operation	REST equivalent	Notes
Query	`GET`	Read. `{ user(id:"123"){ name email } }`
Mutation	`POST/PUT/PATCH/DELETE`	Write. `mutation { createOrder(...){ id status } }`
Subscription	(polling / WebSocket)	Server pushes updates in real time (live order tracking)

GRAPHQL GOTCHAS

Errors return HTTP 200 with details in an `errors` field (message, path). **Caching is hard**: single endpoint + dynamic bodies defeat URL-based HTTP/CDN caching. **Deep nested queries** can melt the server, so enforce depth limits. The backend needs a **resolver** per field; learning curve is real.

gRPC: fast service-to-service calls

RPC = call a function on a remote machine as if local. **gRPC** (Google) = modern RPC on **HTTP/2** with **Protocol Buffers**: binary, tiny, fast, strongly typed.

```
service OrderService {
  rpc PlaceOrder (OrderRequest) returns (OrderResponse);              // unary
  rpc TrackDelivery (DeliveryRequest) returns (stream DeliveryLocation); // server streaming
}
message OrderRequest { string user_id = 1; int32 quantity = 2; }       // numbers = field tags, not
values
```

- **Contract-first:** `.proto` files generate client/server stubs in any language (polyglot).
- **Four modes:** unary (1→1), server streaming (1→many: location feed), client streaming (many→1: bulk upload), bidirectional streaming (chat).
- **Why faster than REST even on HTTP/2:** REST/JSON was designed for HTTP/1.1 text; gRPC uses binary + multiplexing from the start.
- **Limits:** no native browser support (needs gRPC-Web proxy), so browsers use WebSockets; gRPC dominates backend ↔ backend.

Choosing an API style

	REST	GraphQL	gRPC
Format	JSON (text)	JSON (text)	**Protobuf (binary)**
Speed	Medium	Medium	**Highest**
Pattern	Request-response	Request-response + subscriptions	Unary + 3 streaming modes
Contract	Optional (OpenAPI/Swagger)	Strict schema	Strict `.proto`
Use for	Public APIs, simple CRUD, CDN-cacheable data	Many client types, nested data, dashboards	**Internal microservices**, low latency, streaming

Pagination in GraphQL and gRPC

Style	How pagination is expressed
GraphQL	Offset works, but the standard is the **Relay connection** pattern: `products(first: 5, after: "cursor")` (or `last` / `before` backwards) returns a **Connection** = `edges[]` (each with `cursor` + `node`) + `pageInfo` (`hasNextPage`, `endCursor`…). Edges can carry relationship metadata.
gRPC	No query strings; fields live in the messages: request `page_token` + `limit`, response `next_page_token`. Also **streaming pagination**: one server-stream pushes batches; after a disconnect the client resumes from the last cursor.

> **Going deeper →** gRPC internals, mTLS and gRPC vs WebSocket: Ch 21. WebSockets and subscriptions: Ch 8.

8 Real-time communication and webhooks

FOUNDATIONAL

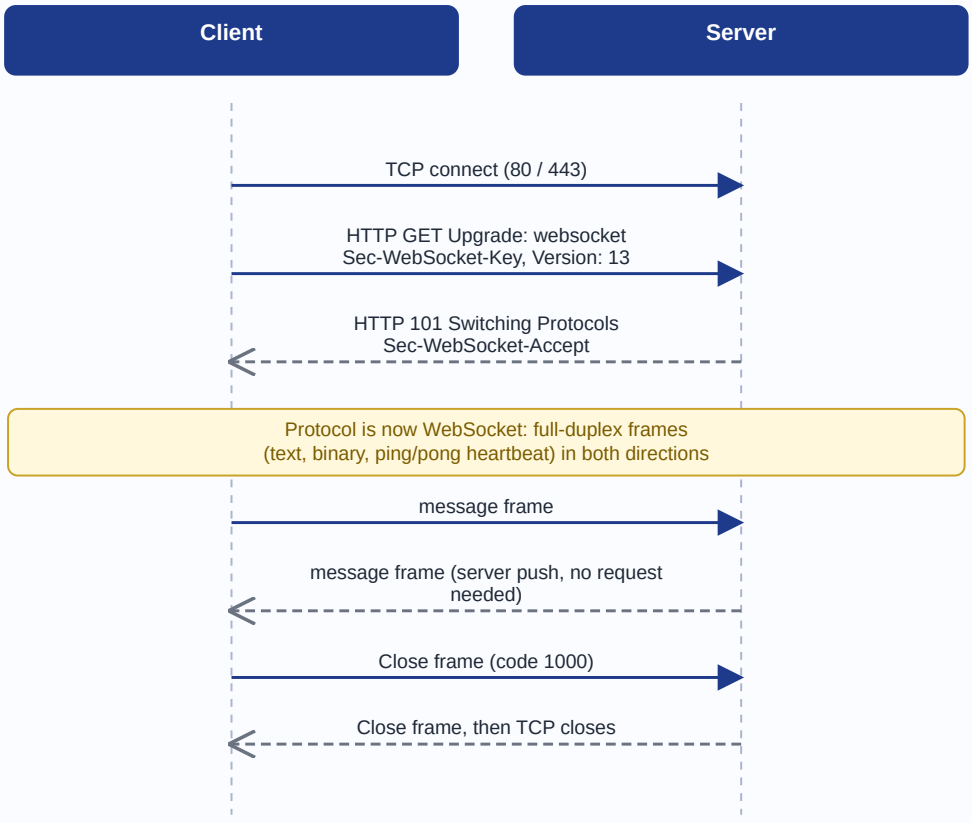
▶ **Source: L28, L16**

Plain HTTP is request-response: the server cannot speak first. Chat, live scores, games, collaborative editing and notifications need data pushed in real time.

The evolution of workarounds

Technique	How it works	Problem
Short polling	Client asks "anything new?" every X seconds	Mostly empty responses: 10M users × every 5 s ≈ **120M requests/minute**; delayed; header overhead
Long polling	Server holds the request open until data or timeout, then client re-requests	Fewer wasted calls but constant connection churn and header cost
SSE (Server-Sent Events)	One persistent HTTP connection; server streams text events	**One-way only** (server → client)
WebSocket	Persistent **full-duplex** connection over one TCP socket	Stateful; needs scaling work (Ch 20)

WebSocket handshake



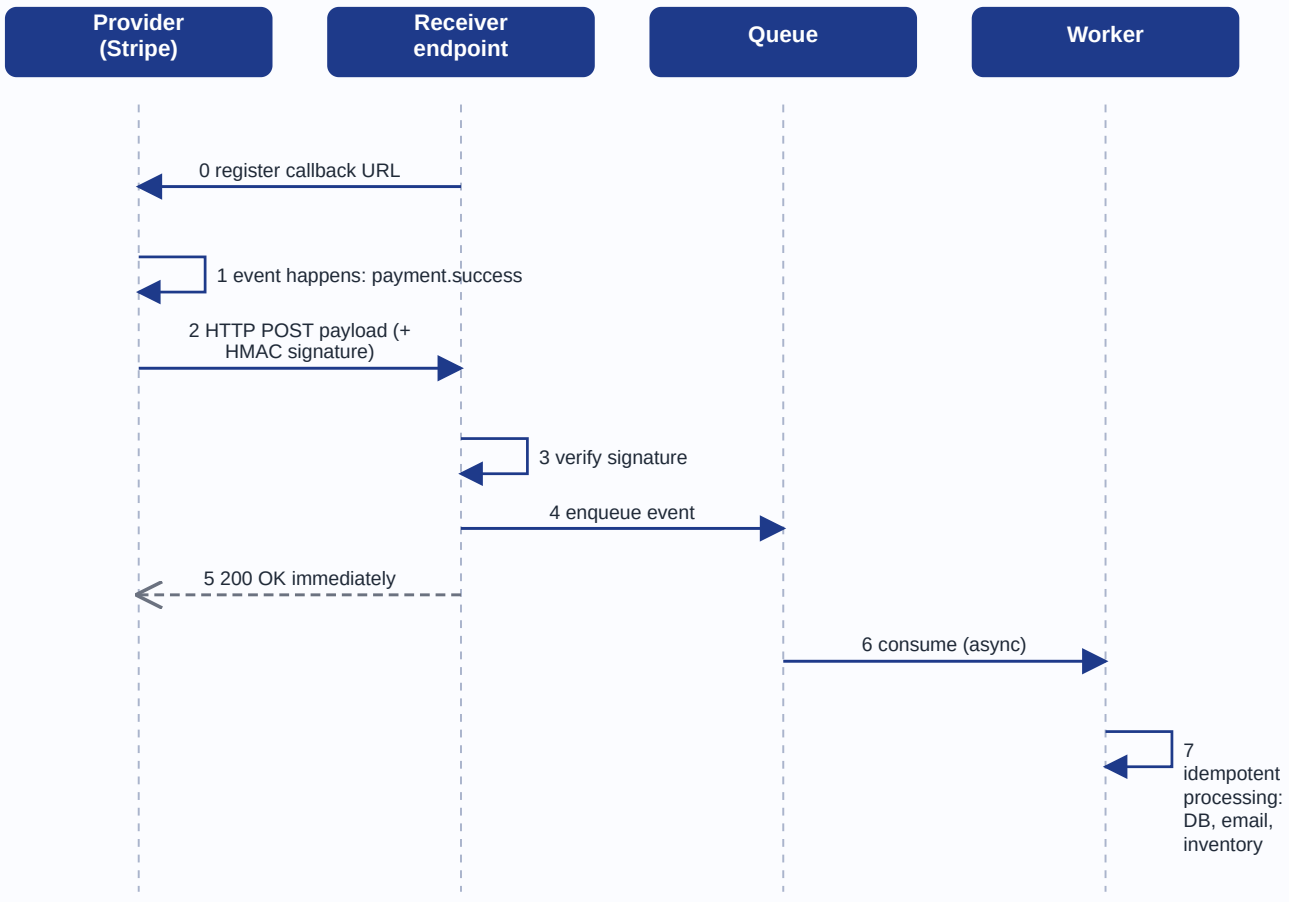
An HTTP request is upgraded (101 Switching Protocols) to a persistent two-way WebSocket; both sides may then send frames at any time.

- URL schemes `ws://` (port 80) and `wss://` (TLS, port 443). Frames: text, binary, ping/pong (heartbeats), close.
- **Graceful close:** either side sends a Close frame (status 1000), the other replies, then TCP closes.
- WebSocket does not guarantee application delivery: add message IDs and app-level **ACKs**, heartbeats, and client auto-reconnect with backoff.

Scenario	Use
Chat, games, collaborative editing, live trading	**WebSockets**
One-way live feeds (tickers, news)	SSE or WebSockets
CRUD, forms, infrequent updates	Plain REST

Webhooks: "don't call us, we'll call you"

A **webhook** is an HTTP callback: when an event happens, the provider `POST`s a JSON payload to a URL you registered. It replaces polling "is it done yet?" with event-driven push. Examples: Stripe/Razorpay payment status, GitHub push triggering CI, Shopify orders, Slack bots.



Reliable webhook receiver: verify, enqueue, return 200 immediately, process asynchronously and idempotently.

Challenge	Mitigation
Receiver down	Provider retries with **exponential backoff**; receiver buffers into a queue
Duplicate delivery	**Idempotency**: dedupe by event/transaction ID

Out-of-order events	Event timestamps or sequence IDs
Spoofed calls	**HMAC signature** with shared secret, auth tokens, IP allow-list
Slow processing → provider timeout	**Acknowledge fast**: 200 OK first, heavy work in a worker; failures go to a **DLQ**

> **Going deeper** → scaling WebSockets across servers and video calls with WebRTC: Ch 20.

9 Sync vs async, message queues and events

FOUNDATIONAL

▶ Source: L14, L15, L17, crash course

Synchronous vs asynchronous

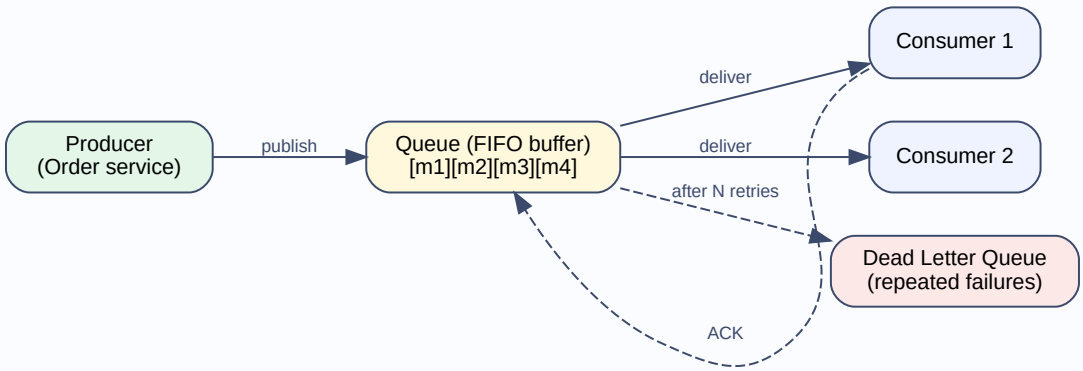
	Synchronous	Asynchronous
Client	Blocks until the response arrives	Fires the request and continues
Analogy	Phone call: you stay on the line	Food order: pay now, cooking/delivery happen in background
Pros	Simple, instant feedback, easy to debug	Fast UX, absorbs spikes, loose coupling, fault isolation
Cons	Latency stacks up, **cascading failures**, threads pile up under load	Eventual consistency, harder tracing, needs retry/DLQ handling
Tech	REST, gRPC unary, GraphQL	Queues (RabbitMQ, SQS), Kafka, webhooks, workers

DECISION RULE

Sync when the caller needs the result now or steps must be strictly ordered (balance check, payment validation).
Async for long/heavy work (video, reports, emails), traffic spikes, or when services must stay decoupled.

Message queue

A buffer between a **producer** and **consumers**: the producer drops a message and moves on; workers process at their own pace. Placing an order returns instantly while email, inventory and logistics happen later.



Producer → queue → consumers. Unacknowledged or repeatedly failing messages end up in a Dead Letter Queue.

Term	Meaning
Message	Small payload (JSON/binary): `orderId`, `userId`, timestamp
Queue	FIFO buffer; older messages first
ACK	Consumer confirms success; only then is the message removed
Dead Letter Queue	Quarantine for messages that keep failing, for debugging

Visibility timeout	(SQS) a picked-up message is hidden for N seconds; if no ACK in time it reappears for another worker

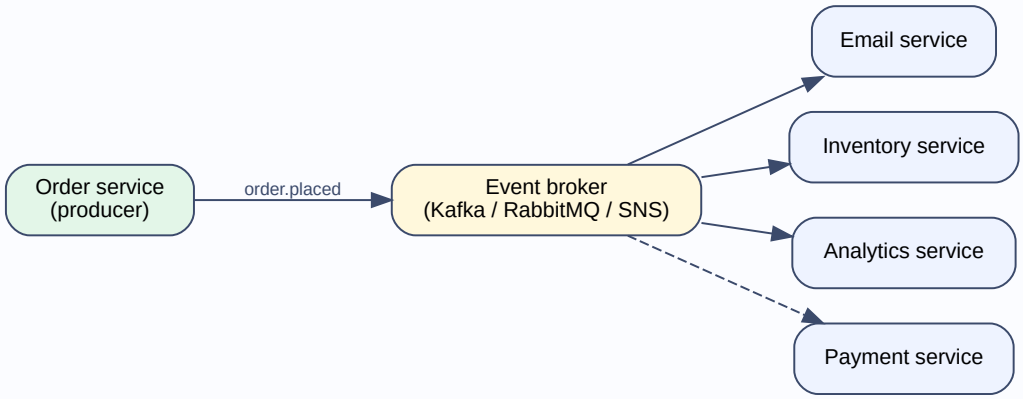
Why interviewers love queues

- **Loose coupling:** producer doesn't care if the consumer is slow or down.
- **Load levelling:** 10 → 1,000 orders/minute is buffered, not dropped.
- **Horizontal scaling:** add consumers to drain a backlog faster (a second worker doubles throughput).
- **Fault tolerance:** messages wait safely while consumers are down; zero loss on restart.

Tools: RabbitMQ (task queue, rich routing), AWS SQS (managed), Apache Kafka (event log, replayable), Redis Streams (lightweight, in-memory).

Event-driven architecture (EDA)

Services communicate by publishing **events** (state changes: `order.placed`, `payment.succeeded`) to a **broker**; interested services **subscribe** and react independently (**pub/sub**).



One event, many independent consumers. Adding a new feature = adding a subscriber; the producer is untouched.

Event style	Meaning / trade-off
Event notification	Event carries only an ID; consumers call back for details. Small payload, extra calls.
Event-carried state transfer	Event carries all needed data (items, price, user). No callbacks; larger payload.

	Request-driven (REST)	Event-driven
Path	Direct point-to-point	Indirect via broker
Coupling	Tight	Loose
Failure impact	Cascades	Isolated; broker buffers
Best for	Synchronous CRUD, direct user requests	Async workflows, microservice fan-out

COSTS TO MENTION

Distributed tracing is needed (Jaeger, Zipkin); **duplicates** are possible so consumers must be **idempotent**; event **ordering** needs partition keys; schemas need versioning; data is **eventually consistent**.

Real-world use

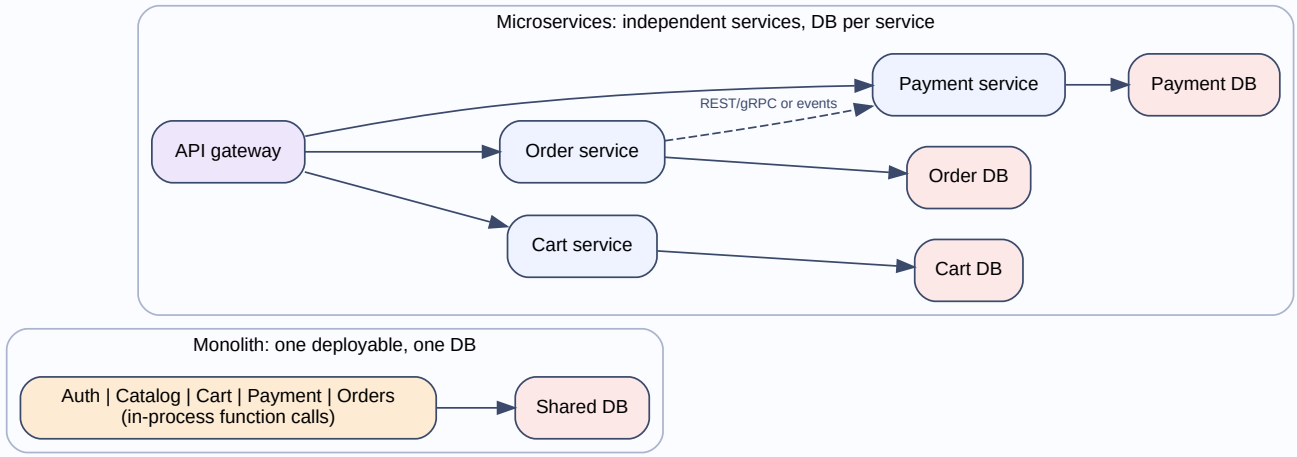
Amazon order → inventory, payment, shipping, email in parallel · bank transaction → fraud detection · Uber ride events → billing, notifications, analytics · Instagram like → notification, feed, recommendations.

> **Going deeper** → RabbitMQ exchanges and Kafka partitions/consumer groups: Ch 16. Event sourcing, CQRS, Saga, outbox: Ch 15.

10 Monolith, microservices and the API gateway

FOUNDATIONAL

▶ **Source: L5, L25, crash course**



Monolith: modules call each other in-process and share one DB. Microservices: independent services, each owning its data, behind a gateway.

	Monolith	Microservices
Deploy	One unit (WAR/JAR/container)	Each service has its own code, pipeline, scaling
Calls	Fast in-process function calls	Network calls (REST/gRPC) or events
Pros	Simple to build, test, deploy; fast; ideal for small teams	**Independent scaling** (scale only Orders on sale day), autonomous deploys, **fault isolation**, polyglot stacks (Java orders, Go payments, Python ML)
Cons	Must scale everything together, merge conflicts, tech lock-in, large **blast radius** (one bug or memory leak can crash all)	Network latency and failures, DevOps overhead (Docker, Kubernetes, service discovery, CI/CD, central logging), distributed data and debugging

MIGRATION ANSWER

"Split by **business capability** (auth, orders, payments, inventory, notifications), not by technical layer. Migrate **incrementally, one service at a time**, put an **API gateway** in front, and give each extracted service its own database. Never a big-bang rewrite."

Data in microservices

- **Database per service (recommended):** independent scaling, loose coupling, mix SQL and NoSQL, fault isolation. Cost: **no cross-service JOINs**; aggregate via API composition or events, and use Saga for multi-service writes (Ch 15).
- **Shared database:** a transitional compromise during decomposition or for reporting.

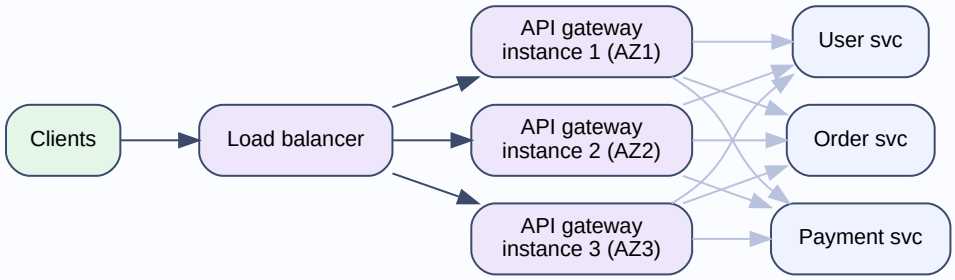
API gateway

Without a gateway, clients make many calls per screen, must know every service URL and contract, security logic (JWT, rate limits, logging) is duplicated per service, and backend changes break clients. An **API gateway** is the

logical single entry point in front of all services.

Responsibility	Detail
Routing	By path, verb, header, region, canary rules
Auth	Validate JWT / OAuth / API key / mTLS once at the edge
Validation	Headers, params, JSON schema (structure only, not business rules)
Versioning	Route `/v1` and `/v2` to different backends during migrations
Rate limiting	Hard (`429`) or soft throttling (Ch 11)
Aggregation (BFF)	Fan out to several services in parallel, return one response
Transformation	gRPC/XML → JSON; strip internal fields
Caching	Cache frequent `GET` s via `Cache-Control`

Flow: client → authenticate + rate-check → validate → route (with timeouts/retries) → transform/cache → respond.
Tools: AWS API Gateway, Azure APIM, GCP, Kong, NGINX, Envoy.



The gateway is a logical entry point but never a single server: stateless instances across availability zones behind a load balancer.

GATEWAY ANTI-PATTERNS

Business logic inside the gateway · over-aggregating dozens of heavy calls into one blocking request · a single unscaled instance (a SPOF) · missing timeouts and retry limits (cascading timeouts).

GATEWAY VS LOAD BALANCER

The LB spreads traffic across identical instances of *one* service. The gateway routes to *different* services and handles cross-cutting concerns (auth, limits, aggregation). A gateway typically forwards to a per-service LB.

11 Authentication, authorization and API security

FOUNDATIONAL

▶ **Source: L7, L32, L33, OAuth 2.0 lecture, security notes**

AUTHN VS AUTHZ (AIRPORT ANALOGY)

Authentication = *who are you?* (showing your ID and ticket). **Authorization** = *what may you do?* (does an economy ticket allow the business lounge?). Failed authN → `401` . Authenticated but not allowed → `403` .

Why we need sessions at all

HTTP is **stateless**: the server doesn't remember who sent the previous request, and sending the password every time is insecure. So we authenticate once and give the client something to prove it later: a **session ID** or a **token**.

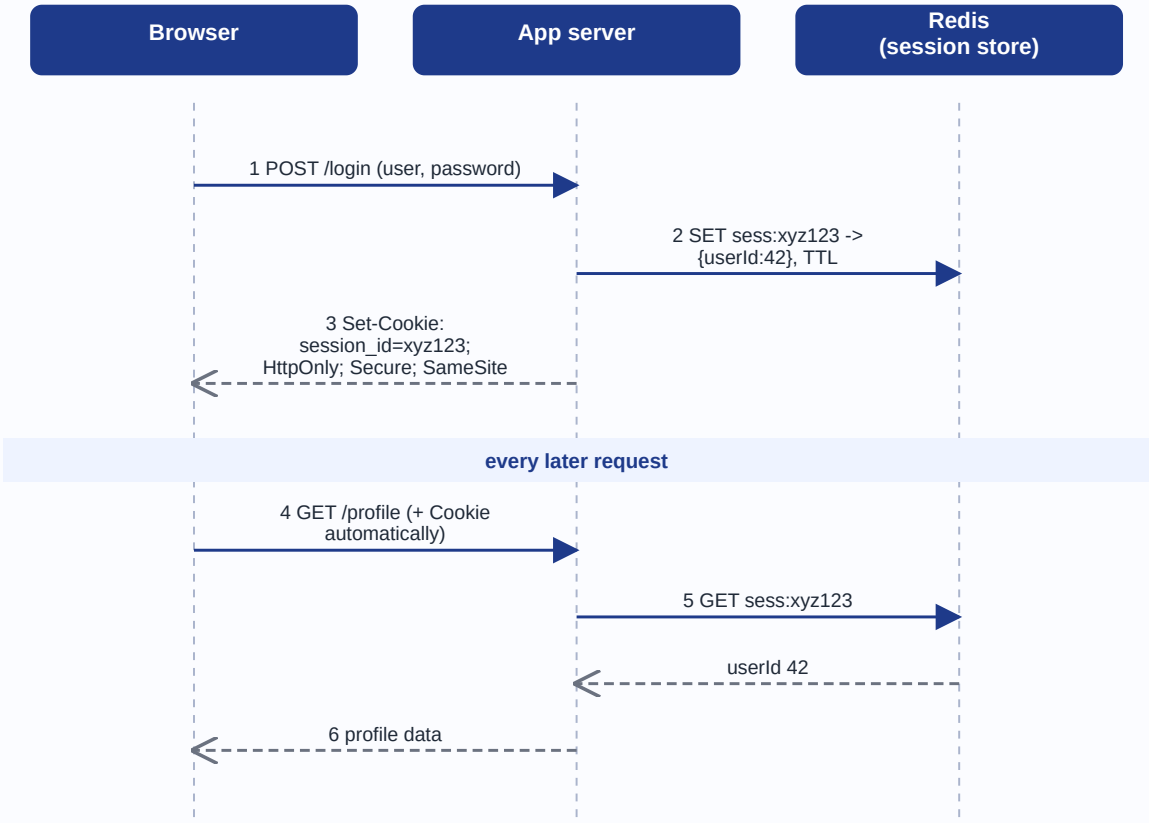
Authentication methods

Method	How	Notes
API key	Unique key in header/query per developer or service	Simple; weak for public end-user apps; fine for internal service-to-service
Basic	`username:password` Base64-encoded in `Authorization` on every call	Base64 is *encoding*, not encryption. Safe only over HTTPS
Session / cookie	Server stores session, browser holds session-ID cookie	**Stateful**
JWT / Bearer	Signed token carries identity and claims	**Stateless**; theft = access until expiry
OAuth 2.0 / OIDC	"Login with Google", delegated scoped access	OAuth = authorization framework; **OIDC** = identity layer on top; SSO via SAML (XML) or OIDC (JSON)
mTLS	Both sides verify certificates	Banking, zero-trust microservices (Ch 21)

ENCODING VS HASHING VS ENCRYPTION

Encoding (Base64, URL): reversible by anyone, for compatibility, no secret. **Hashing** (SHA-256, bcrypt, Argon2): one-way, for integrity and passwords; use a **salt** for password hashing; same input → same output; the output length is fixed. **Encryption**: reversible with a key. A bearer token is just a string whose holder gets access: treat it like a password.

Session-based authentication



Login creates a server-side session; the cookie is only a pointer (the session ID), never user data.

- **Scaling problem:** sessions in one server's RAM fail behind a load balancer (request 2 lands on a server without the session). **Sticky sessions** work but limit auto-scaling and lose sessions if that server dies.
- **Industry standard:** a shared **Redis** session store. Why not SQL? Access pattern is write-once, **read on every request**: Redis gives sub-ms lookups and native **TTL** expiry.
- **Logout/revocation is easy:** delete the key and the next request gets 401.

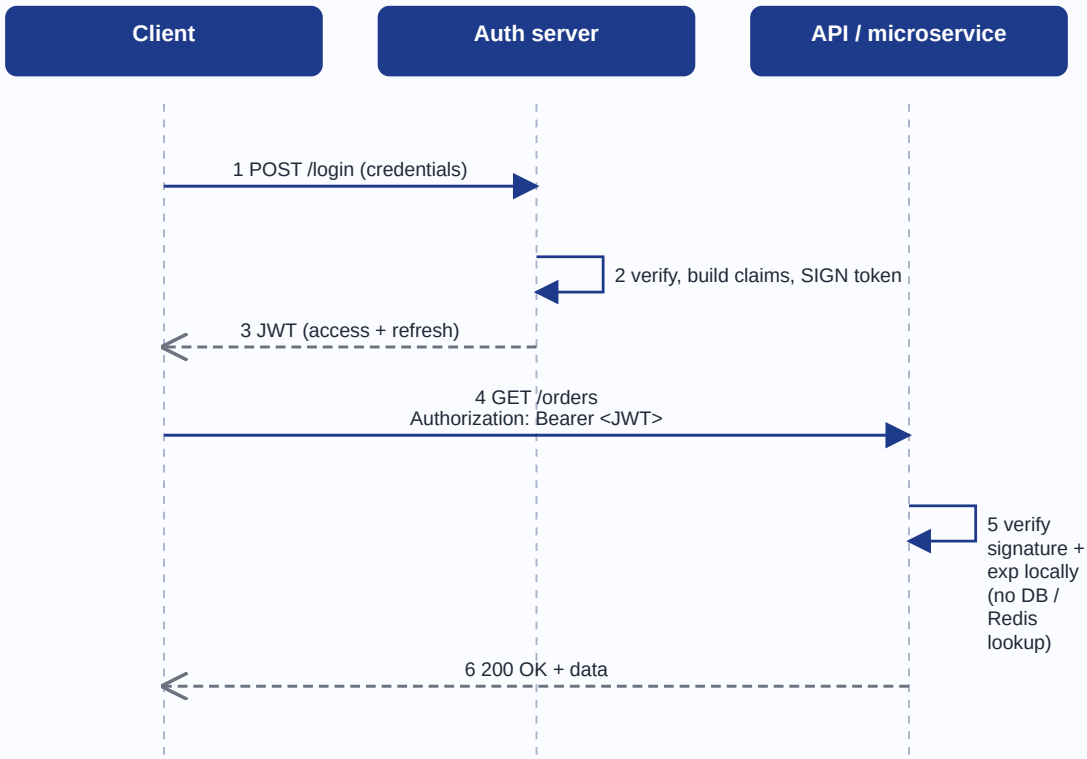
Attack	Defence
Session hijacking (sniffing, XSS cookie theft)	`Secure` flag (HTTPS only), `HttpOnly` (JS can't read it), short TTL (5–10 min for banking)
Session fixation	**Regenerate the session ID** right after login
CSRF (a malicious site makes your browser send authenticated requests)	`SameSite=Strict/Lax` cookies + CSRF tokens on state-changing requests
Session prediction	Cryptographically random IDs (UUID v4)

JWT: stateless tokens

Session auth needs a store lookup on every request and shared infrastructure across services; cookies are awkward for mobile apps. A **JWT** lets any service verify identity **locally, with no lookup**.

HEADER	PAYLOAD	SIGNATURE
algorithm (HS256 / RS256), type	claims: userId, roles, exp, iat	sign(base64(header).base64(payload), key)

- Three Base64URL parts joined by dots. The payload is **readable by anyone** (signed, not encrypted): **never put secrets in it**. Tampering invalidates the signature.
- **Verification:** split → check algorithm (enforce the expected one, never trust the token's own header) → recompute signature → compare → check `exp`, issuer, audience. Zero DB calls.



JWT flow: the API verifies signature and expiry locally.

	Sessions	JWT
State	Stateful (server remembers)	Stateless (client carries it)
Per-request lookup	Required (Redis/DB)	None
Scaling	Needs shared store	Natural fit for microservices and APIs
Revocation	**Easy**	**Hard**: short TTL + refresh-token revocation
Size	Tiny ID	Larger token
Best for	Server-rendered web apps, banking, admin panels (instant logout)	Microservices, mobile, REST APIs, OAuth/SSO

ACCESS + REFRESH TOKENS

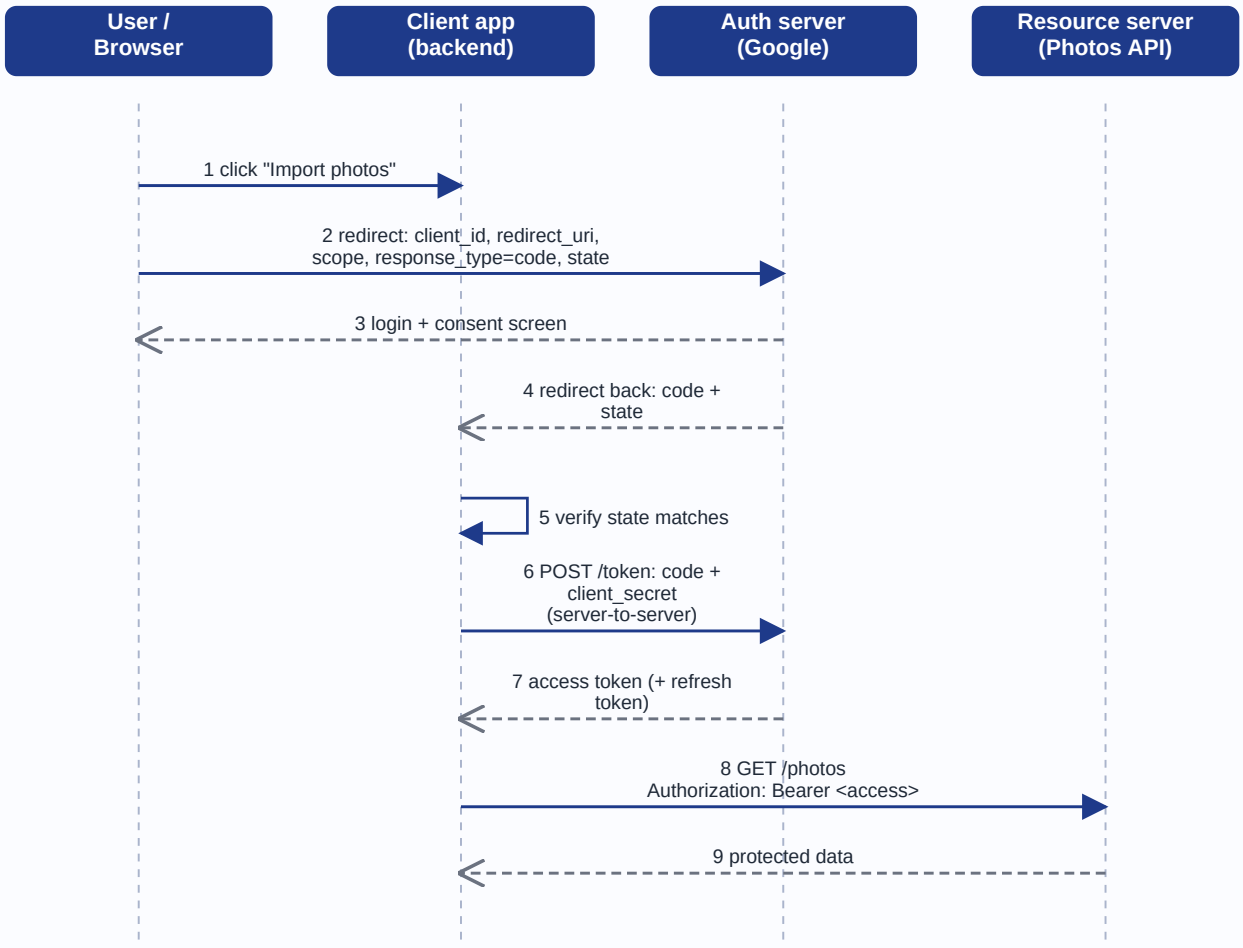
Short-lived **access token** (5–15 min) calls APIs. Long-lived **refresh token** (stored server-side and in an `HttpOnly`, `Secure` cookie) gets new access tokens without re-login. The auth server is **not a SPOF for normal traffic**: services verify JWTs locally, only new logins fail while it's down. Verify JWTs once at the API gateway.

OAuth 2.0: delegated authorization

Problem: to let an AI photo editor read your Google Photos you used to hand it your Google password: full account access, leak risk, and revoking meant changing your password. **OAuth 2.0** issues the app a **scoped, temporary access token** instead.

Role	Example
Resource owner	The user who owns the photos
Client	The third-party photo editor
Authorization server	Google OAuth server (authenticates, takes consent, issues tokens)
Resource server	Google Photos API (accepts access tokens)

Registration first: the developer registers the app and receives a public **client_id**, a secret **client_secret** (backend only; never in SPAs/mobile code) and a whitelist of **redirect URIs**. Request parameters: `client_id`, `redirect_uri`, `scope` (`photos.readonly`), `response_type=code`, and `state` (random value that defends against CSRF).



Authorization Code flow. The browser only ever sees a short-lived, single-use code; the token travels server-to-server.

WHY A CODE, NOT A TOKEN, IN THE REDIRECT?

Tokens in browser URLs leak via history, referrers and logs (the old Implicit flow). The one-time code is useless without the `client_secret` (or PKCE verifier) used in the back-channel exchange.

	Access token	Refresh token
Role	Calls protected APIs	Gets new access tokens
Lifetime	Short (15–60 min)	Long (days) or until revoked
Sent to	Resource server (`Bearer`)	Authorization server only

Authorization models

Model	Rule	Example
RBAC	Permissions by **role**	GitHub: Admin deletes repo, Editor pushes, Viewer reads
ABAC	Fine-grained rules on **attributes**	Department = HR AND age > 18
ACL	Permissions attached to the **resource**	Google Drive: share this doc with Alice as read-only

Typical flow: `POST /login` → token with role → `GET /profile` as User = 200; `GET /admin/report` as User = **403**, as Admin = 200.

Seven security techniques

Technique	Purpose
Rate limiting	Stops brute force and DDoS: cap requests per IP/user
CORS	Restrict which origins' browsers may call your API
Injection prevention	Parameterised queries stop SQL injection
WAF (firewall)	Filters known attack patterns
VPN	Keep admin dashboards on a private network
CSRF tokens	Banking example: a malicious page can't trigger a hidden transfer with your cookie
XSS protection	Sanitise user content (a comment can't run JavaScript in others' browsers)

Rate limiting algorithms

Enforced at the API gateway; counters usually live in Redis (`INCR` + `EXPIRE`); rejected calls get `429 Too Many Requests` . **Hard limit** rejects immediately; **soft throttling** slows requests down.

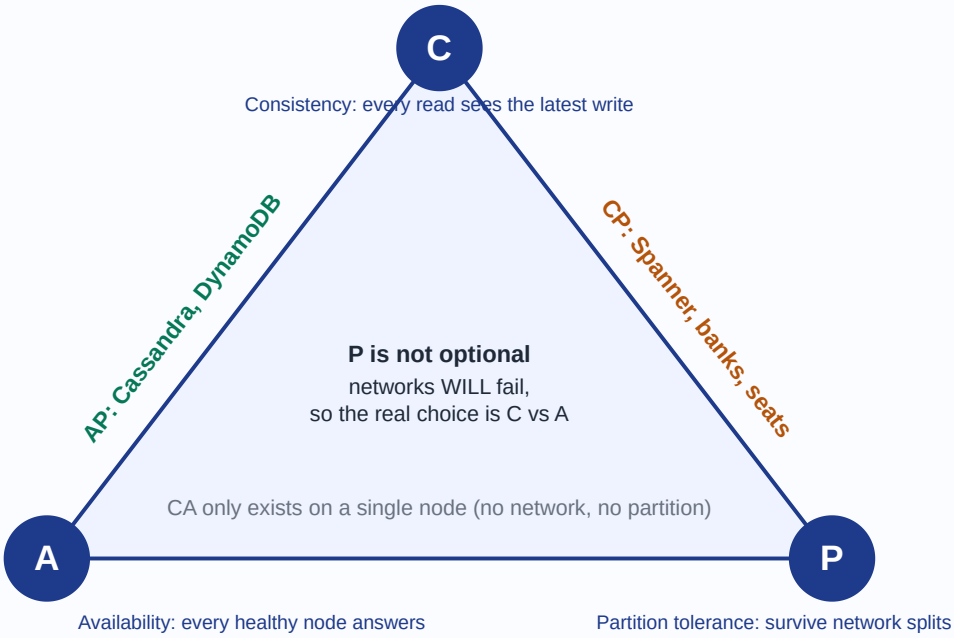
Algorithm	Idea	Trade-off
Token bucket	Bucket refills at a steady rate; each request takes a token	Allows short bursts up to bucket size
Leaky bucket	Requests queue and drain at a constant rate	Smooth output; no bursts
Fixed window	Counter per time window (e.g. 100 per minute)	Simple; boundary burst (200 around the window edge)
Sliding window	Counts over a rolling interval	Accurate; costs more memory/CPU

> **Going deeper →** PKCE for SPAs/mobile, JWT signing algorithms (HS256 vs RS256), refresh-token rotation and mTLS: Ch 21. Redis rate limiter: Ch 19.

12 CAP theorem and consistency FOUNDATIONAL

▶ Source: L24

In a distributed data store you can guarantee at most **two of three**: **C**onsistency (every read gets the latest write or an error), **A**vailability (every healthy node answers, possibly with stale data), **P**artition tolerance (keeps working when messages between nodes are lost).



CAP: since networks do fail, P is mandatory; the real choice during a partition is C or A.

Choice	Behaviour in a partition	Pick when
AP	Keep serving; may return stale/conflicting data (eventual consistency)	Uptime matters more than freshness: social feeds and bios, Netflix descriptions, product reviews, restaurant menus
CP	Refuse or error rather than serve stale data	Stale data causes loss or legal risk: seat booking, flash-sale inventory (no overselling), bank ledgers, payments

DON'T CONFUSE THE TWO "CONSISTENCIES"

	CAP consistency	ACID consistency
Scope	Across **nodes**: all replicas show the same latest value	Inside **one DB transaction**: valid state to valid state, constraints hold (no negative balance, no foreign-key violation)

Consistency models (strongest → weakest)

Model	Guarantee
Strong	Reads immediately return the latest write everywhere; higher write latency
Bounded staleness	Stale only within a bound (≤ 5 s or ≤ 2 versions)
Read-your-writes	You always see your own updates, even if others briefly don't
Causal	Causally related events appear in order for everyone (a reply never appears before its comment)

Eventual	With no new writes, replicas converge over time (Cassandra, DynamoDB default)

> **Going deeper →** PACELC (what happens when there is *no* partition) and Lamport clocks: Ch 22. Quorums and split-brain: Ch 15.

13 Back-of-the-envelope estimation FOUNDATIONAL

▶ **Source: estimation worked example**

FRAMEWORK OVER PRECISION

Interviewers judge your **structure and assumptions**, not the exact number. State assumptions, convert daily totals into **per-second** rates (what machines actually process), then size storage, cache and servers.

Standard conversions (memorise)

Fact	Rule of thumb
Seconds in a day	86,400 ≈ $\mathbf{10^5}$
Per-second rate	requests per day ÷ 86,400
Size units	1 KB = 10^3 B · 1 MB = 10^6 · 1 GB = 10^9 · 1 TB = 10^{12} · 1 PB = 10^{15}
Peak load	Assume a peak multiplier over average (commonly 2–3×) and say so
Storage	items × size × replication factor (+ extra copies such as transcoded qualities)

Worked example: YouTube daily upload storage

Step	Calculation
Assumptions	1 B users; 1 in 1,000 uploads daily → 10^6 uploaders; avg video 10 min → 10^7 video-minutes/day
Raw size	10^7 min × ~3 MB/min (web-quality) = **30 TB/day**
Replication (×3)	→ **90 TB**
Multiple resolutions (+~×2)	1080p … 144p → **~180 TB ≈ 0.2 PB/day**
Ingest throughput	180 TB ÷ 86,400 s ≈ **~400 MB/s**
Workers	If read + transcode + write of 1 MB takes 50 ms → 20 MB/s per worker → **~20 cores/nodes**
Cache (thumbnails)	90 days × 10^6 ≈ 90 M videos × 10 KB = **~900 GB ≈ 1 TB**; with 16 GB nodes at 50% target fill → **~500 nodes**

SAY THIS

"I'll assume X, Y, Z. That gives N requests/second, M TB/day. I'll size for 3× replication and peak load, and round aggressively because the goal is the order of magnitude."

PART 2

Part 2 · Advanced

Senior-level depth: sharding and hashing internals, consensus and distributed transactions, Kafka and RabbitMQ, storage engines, specialised data stores, Redis at scale, real-time media and advanced security. **Skippable if time is short.** Chapters are independent.

14 Database sharding and consistent hashing

ADVANCED

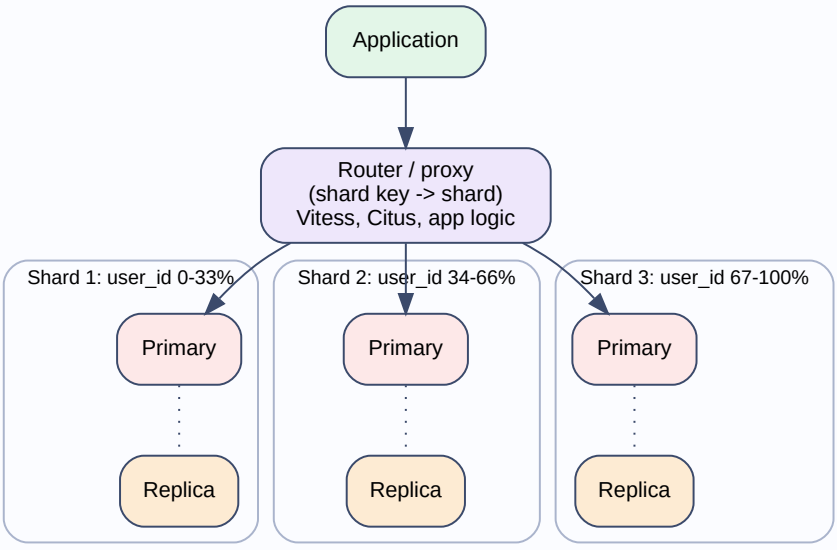
▶ Source: L21, L22

Why shard

A single database eventually hits the vertical-scaling wall: hardware ceilings, disk I/O, query latency. **Sharding** horizontally splits one dataset across **multiple independent database instances** (shards), each with its own CPU, RAM and storage. Together they hold the full dataset.

	What it does	Solves
Table partitioning	Splits a table into chunks *inside one DB engine*	Query pruning; **not** hardware limits (same CPU/RAM/disk)
Replication	Copies the *same* data to several nodes	**Read** throughput and fault tolerance; not write/storage limits
Sharding	Splits *different* data across engines	**Write capacity and storage** beyond one machine

Components: shard (an independent DB), **shard key** (column deciding placement, e.g. `user_id`), **shard map/directory** (key or range → server) and a **router** that inspects the key and forwards the query.



Each shard is itself a small cluster (primary + replicas) so one machine failure never takes a slice of users offline.

Choosing a shard key

Criterion	Bad example
High cardinality (many unique values)	`is_active` : 2 values = max 2 shards
Uniform distribution	`country_code` when 80–90% of users are in one country
Stable (rarely changes)	`email` : every change migrates the user's data across DBs

Matches query patterns	Key unrelated to common queries → expensive scatter-gather

Strategies

Strategy	Mechanics	Pros	Cons
Range	Contiguous ranges (IDs 0–5M → shard 1)	Simple; efficient range scans	**Hotspots** with sequential keys/timestamps (newest range gets 95% of traffic)
Hash	`hash(key) % N`	Even distribution	Changing N remaps ~everything → use consistent hashing
Directory	Lookup table key → shard	Flexible; dedicated shards for VIPs; manual rebalancing	Extra lookup hop; directory is a potential SPOF (cache + replicate it)
Geo	By region (India → Mumbai, EU → Frankfurt)	Low latency, data residency	Cross-region queries and user migration are hard

Challenges and fixes

Problem	Mitigation
Hot shard (celebrity: 900k reads/s vs 5k for normal users; timestamp clustering)	Monitor per-shard QPS/CPU; **dedicated VIP shards** via directory; **compound keys / salting** (`creator_id + day`) to scatter one creator's data
Cross-shard queries (scatter-gather: slow, partial failures)	Cache results in Redis with short TTL; **pre-compute** aggregates in background; **denormalise** hot fields
Cross-shard transactions (debit on shard 1, credit on shard 2)	Design so related data shares a shard (`user_id`); otherwise **Saga** (eventual consistency). 2PC is correct but slow and rarely used at scale (Ch 15)

Routing: application-level (logic + connection pools in code) or proxy/middleware (Vitess for MySQL, Citus/ShardingSphere for PostgreSQL).

> **THE GOLDEN RULE: DON'T SHARD ON DAY 1**
> Sharding adds huge complexity (migrations, backups, joins, transactions). A single managed instance goes far: Aurora Postgres scales to **256 TB**, 128 vCPUs, 768 GB RAM and 256k IOPS; one primary handles ~10–20k writes/s and replicas serve millions of reads/s. First use **caching, read replicas, vertical scaling, indexes**. Shard only when write throughput or storage exceeds the biggest instance.

Zero-downtime migration to a sharded cluster

1. Pick shard key and routing strategy (e.g. hash + consistent hashing).
2. Provision the sharded cluster and routing layer.
3. **Backfill** historical data with background ETL.
4. **Dual-write** to old and new systems.

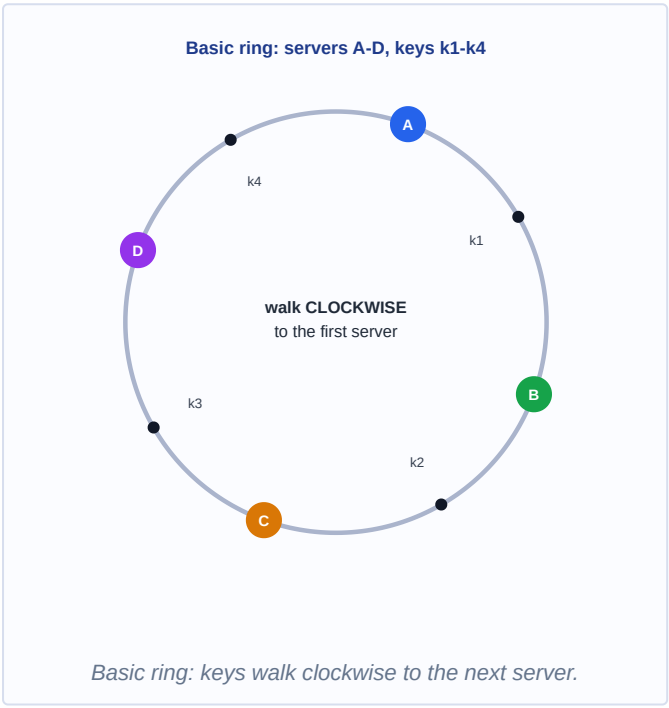
5. Verify parity, **cut reads over**, decommission the old DB.

Consistent hashing

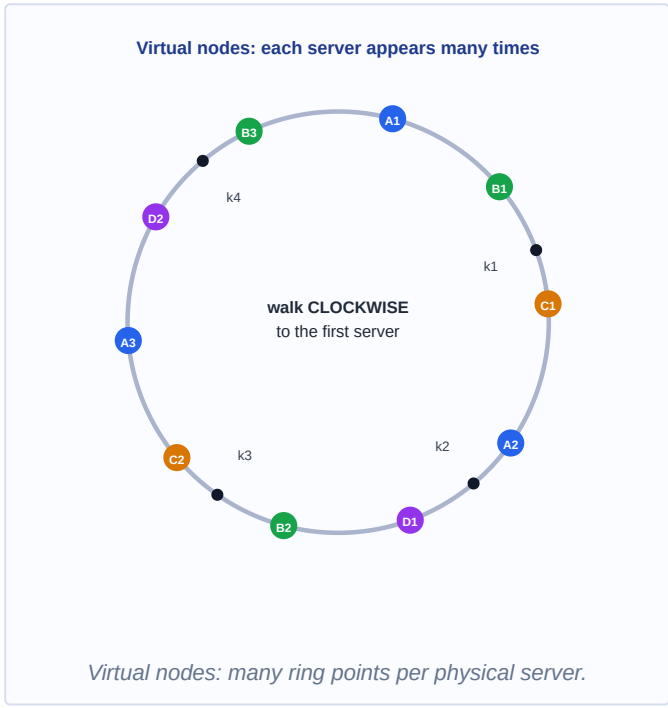
Problem with modulo hashing: `server = hash(key) % N`. When N changes, almost **100% of keys** remap, causing mass data migration, cache invalidation storms, DB overload.

Hash ring: hash space 0…2^{32}–1 bent into a circle. Hash servers and keys with the *same* function; a key belongs to the **first server clockwise** from its position (wrapping past the end to 0).

- **Add a node:** it takes over only the key range between its predecessor and itself (~**1/N** of keys). Everything else stays put.
- **Remove a node:** only its keys move to the next clockwise server.



Basic ring: keys walk clockwise to the next server.



Virtual nodes: many ring points per physical server.

Virtual nodes (vnodes)

With one point per server the ring segments are uneven (hotspots) and a dead server dumps its whole load on one neighbour (**cascading failure**). Give each physical server **many virtual points** (100–200 typical; Cloudflare 256–512) placed at `hash(ServerName + index)` so every client builds an **identical ring deterministically**. A failed server's load spreads across *all* others. Stronger machines get **more vnodes** (300 vs 100 = 3× traffic).

Replication on the ring: store each key on the next **R distinct physical** servers clockwise, skipping extra vnodes of the same server, so a crash fails over to a replica.

> **WHEN TO MENTION IT**
>
> Overview round: "Cassandra, DynamoDB, Redis Cluster and CDNs use consistent hashing internally." Deep-dive round: design it yourself when asked to build a distributed cache, key-value store, message queue or sharding proxy.

15 Replication, consensus and distributed transactions ADVANCED

▶ Source: replication and consensus notes, distributed transactions video, L21

Replication modes

	Behaviour	Risk
Asynchronous	Primary acknowledges, then ships changes; low write overhead	**Replication lag**; if the primary dies before syncing, data is lost/inconsistent
Synchronous	Replicas apply the same operation log in the same order before commit	Higher write latency; blocks if a replica is slow

	Master-slave (primary-replica)	Master-master (multi-master)
Writes	Only the master	Multiple nodes
Pros	Simple, safe ordering	Better write availability and resilience
Cons	Write bottleneck, failover needed	**Split brain** and conflicts

SPLIT BRAIN AND QUORUM

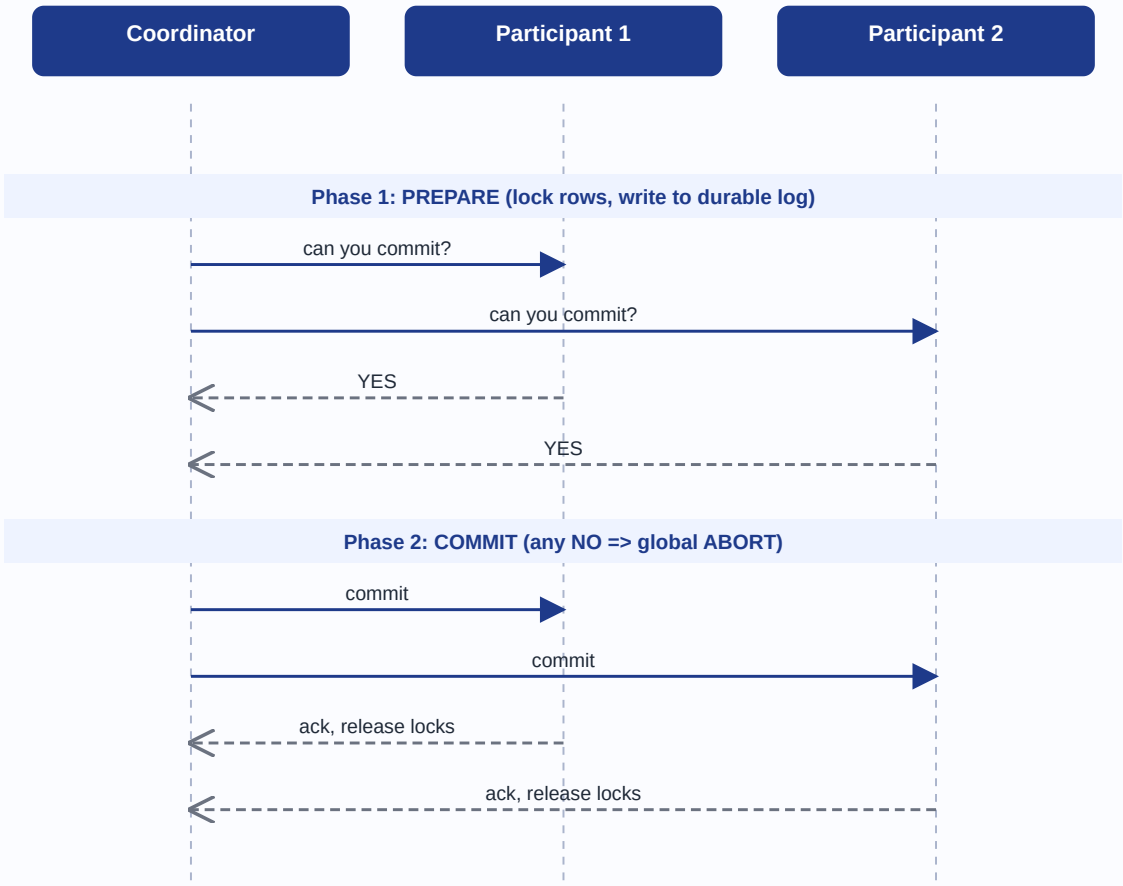
During a **network partition** two masters each assume the other died and accept conflicting writes (two deductions → negative balance). **Fix: quorum / majority.** With 3 nodes a write commits only if a **majority (2)** agrees; the isolated minority cannot commit, and its uncommitted state is rolled back to the majority's. This is the idea behind consensus protocols (Raft, Paxos).

MVCC (PostgreSQL): keeps multiple row versions so readers and writers don't block each other and isolation levels can be tuned (dirty read … serializable).

Distributed transactions

In a monolith, ACID rolls back automatically. Across microservices or shards, each database is independent and no single lock covers them.

Two-Phase Commit (2PC)

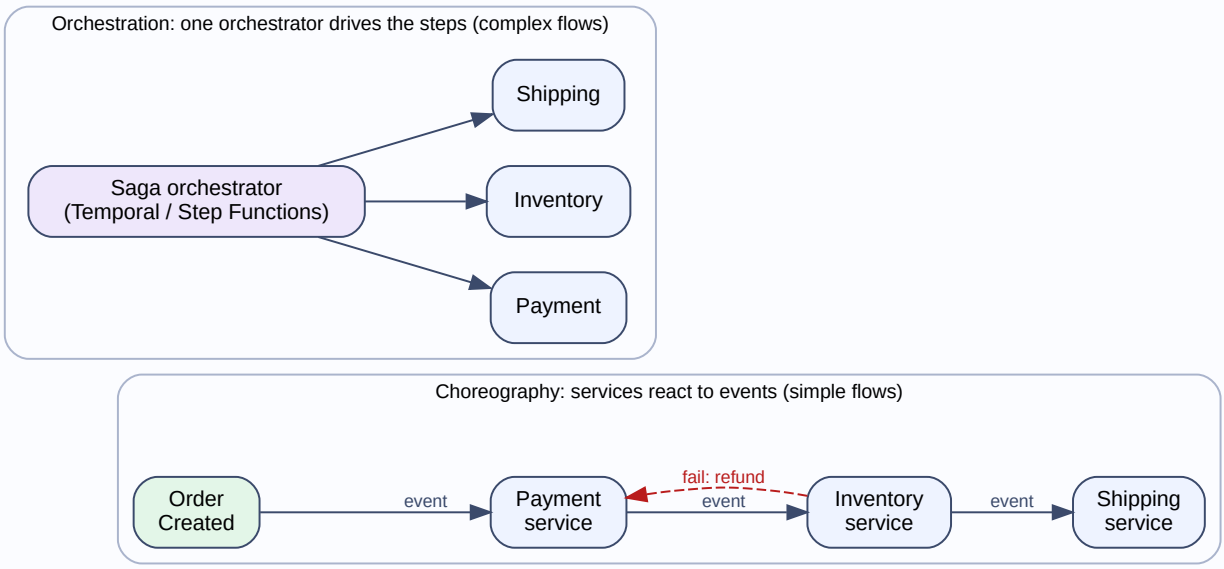


A coordinator asks every participant to prepare (lock + log), then commits only if all said YES.

- **Blocking:** if the coordinator or a participant crashes after voting, rows stay locked; throughput drops to the slowest participant.
- **Fit:** inside tightly coupled distributed databases (Google Spanner, YugabyteDB), **not** across independent microservices.

Saga pattern

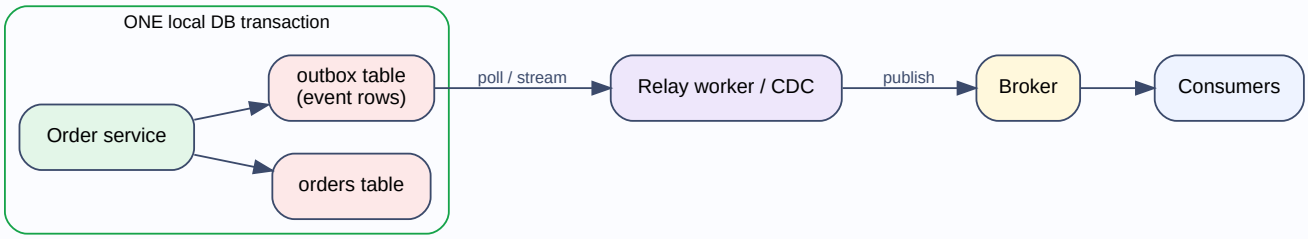
Replace one big blocking transaction with a chain of **local transactions** and **eventual consistency**. If step N fails, run explicit **compensating actions** (business-level undo: issue a refund, not a DB rollback).



Choreography (services react to events; good for 2–3 steps) vs orchestration (a central orchestrator such as Temporal / AWS Step Functions drives and handles failures).

Transactional outbox: the dual-write fix

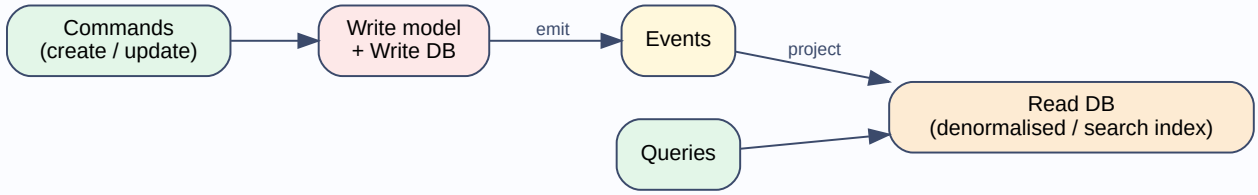
Writing to the DB *and* publishing an event are two systems; one can succeed while the other fails. Write the data **and** the outgoing event row in **one local transaction**; a relay worker or **CDC** publishes the outbox rows.



The event is stored atomically with the data, then published reliably.

Event sourcing and CQRS

- **Event sourcing:** store an **append-only log of every state-changing event** instead of just the current row. Current state = **replay** of events (a balance = sum of deposits and withdrawals). Immutable, audit-friendly.
- **CQRS:** separate the **write model** (commands on a transactional DB) from the **read model** (queries on a denormalised DB/search index kept in sync via events); scale and index each independently.



CQRS: commands update the write DB; events project into a read-optimised store.

> **DECISION RULE**
> "2PC *inside* a distributed database; Saga + outbox *across* services. And wherever possible, design so one transaction stays on one shard."

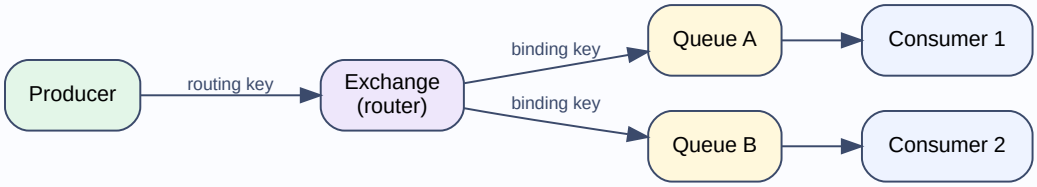
16 Messaging in depth: RabbitMQ and Kafka

ADVANCED

▶ **Source: L17, L18, L19**

RabbitMQ: the smart broker

Broker speaking **AMQP** (binary). Producers publish to an **exchange** (never directly to a queue); the exchange routes to queues by **bindings**; consumers read queues and ACK. Post-office analogy: sender → post office (exchange) → mailbox (queue) → recipient.



Producer → exchange (routing brain) → bound queues → consumers.

Exchange	Routes by	Use
Direct	Binding key == routing key exactly	`error` / `info` log queues
Fanout	Broadcast to **all** bound queues; keys ignored	Notifications, alerts, `order.placed` to many services
Topic	Wildcards: `*` = exactly one word, `#` = zero or more (`order.*` ≠ `order.created.success` ; `user.#` matches both)	Geo / multi-level event categories
Headers	Header key-values; `x-match: all|any`	Attribute-based routing

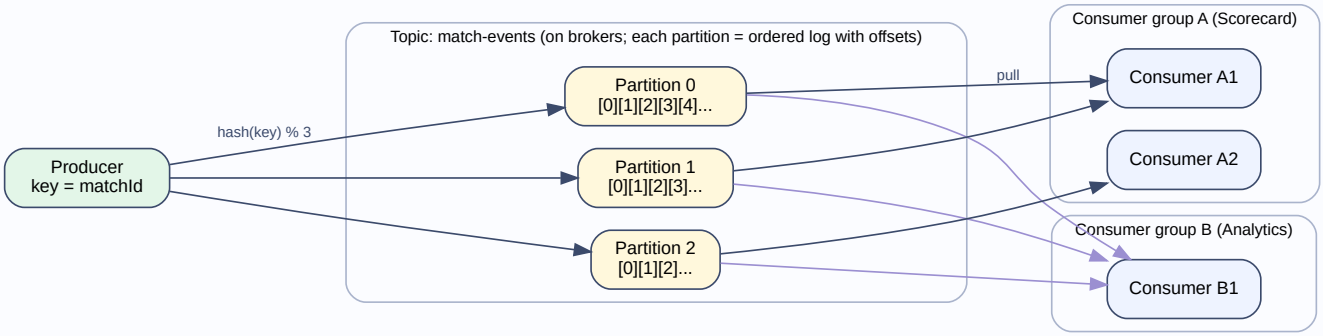
Queue type	Behaviour
Priority (`x-max-priority`)	High-priority messages (VIP orders) jump ahead
Dead letter (DLX)	Failed/expired/rejected messages diverted instead of looping forever
Lazy	Writes to disk, not RAM: low memory during large backlogs, slightly more I/O latency
TTL (`x-message-ttl`)	Unconsumed messages expire
Single active consumer	One consumer at a time for strict order; standby takes over on failure
Stream	Append-only log, replay by offset (Kafka-like)

- **Connection vs channel:** a TCP connection is heavy; a **channel** is a cheap virtual connection multiplexed inside it. **Virtual hosts** isolate environments (dev/prod) in one broker.
- **Reliability:** `ACK` deletes the message; `NACK requeue=true` retries; `NACK requeue=false` → DLQ/drop. Durable queue + persistent message survive restarts. A single node is a SPOF: use **clustering and quorum queues**.

Kafka: the distributed event log

> **WHY NOT REST CALLS OR A DATABASE AS THE QUEUE?**
> Direct REST from producer to every consumer = tight coupling, cascading failures, ordering bugs, data loss during outages, no scalability. A DB as a queue = polling delays, lock contention, non-atomic commit timestamps breaking order, and thousands of writes/s versus the millions needed.

Kafka is a distributed event streaming platform: millions of messages/s, disk-backed durability, replayability.



Producer partitions by key; each partition is consumed by exactly one consumer per group; different groups read independently.

Concept	Meaning
Record	Key, value, timestamp, headers
Topic	Named category (`order-events`)
Partition	Ordered, immutable log; each record gets an **offset**. **Order is guaranteed only within a partition**
Partitioning	**With key:** `hash(key) % partitions` (same `matchId` / `orderId` → same partition → ordered). **No key:** round robin
Broker / cluster	Server holding partition logs; several brokers = cluster
Leader / follower / ISR	Each partition has a leader (all reads/writes) and followers; if the leader dies an **in-sync replica** is promoted
Zookeeper → KRaft	Legacy Kafka used external ZooKeeper for metadata/controller election; modern Kafka uses built-in **KRaft**

Consumers

- **Pull model:** consumers fetch batches at their own pace.
- **Consumer group rule:** each partition is read by **only one consumer in a group**; different groups (Scorecard, Analytics) each get all data independently. Partitions cap parallelism: more consumers than partitions = idle consumers.
- **Rebalancing:** when a consumer dies, its partitions are redistributed to the rest of the group.
- **Offsets** are committed to `__consumer_offsets` ; restart resumes from the last commit. A new group with `fromBeginning` **replays history** from disk.

Producer `acks`	Guarantee
`0`	Fire and forget: fastest, can lose data

`1`	Leader wrote it: balanced
`all`	Leader + all in-sync replicas: safest

Production issues

Issue	Fix
Hot partition (viral ad ID, mega-merchant)	**Compound keys** (`AdID + region`), **key salting** (random suffix 1..N), or no key (round robin)
Large payloads (Kafka likes < 1 MB)	Store the file in S3; put the object URL in the message
Failures	Producer retries, **retry topic**, then **DLQ** for permanent failures
Retention	By time (`retention.ms` , default 7 days) or size (`retention.bytes`)

	RabbitMQ	Kafka
Model	Smart broker, simple consumer (task queue)	Simple broker, smart consumer (log)
Routing	Very rich (4 exchange types)	Partition-key based
Retention	Deleted after ACK	Kept by time/size; replayable
Throughput	~50–100k msg/s per node	Millions msg/s per cluster
Pick for	Background jobs, complex routing	Event streaming, analytics, replay, multiple independent consumers

Other brokers: AWS SNS + SQS (managed **fan-out**: SNS topic → many SQS queues), Google Pub/Sub, Redis Streams (in-memory, low latency).

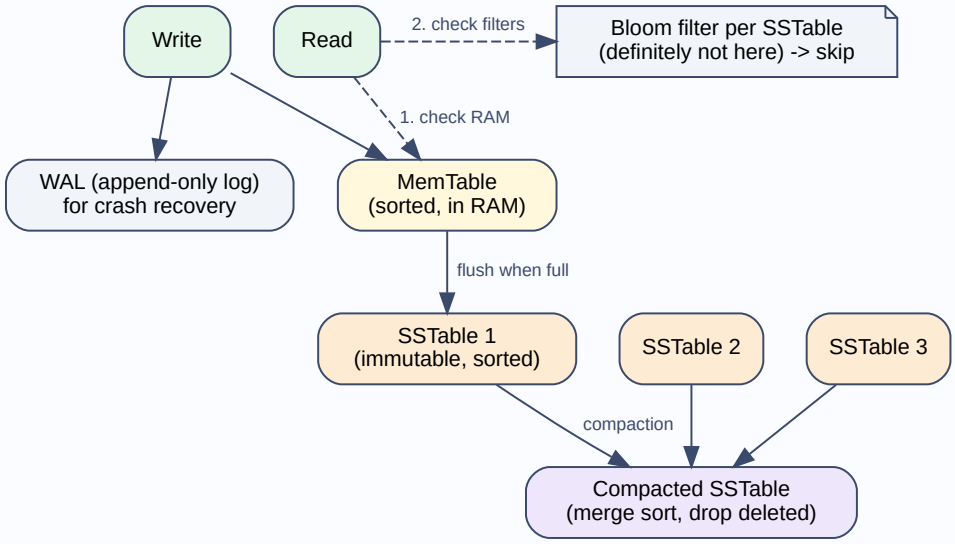
> **QUEUE VS STREAMING**
> Use Kafka as a **queue** for massive-scale task decoupling (100k–1M+ msg/s where RabbitMQ/SQS strain); as a **stream** when several downstream systems need the same continuous real-time events (driver locations, live scores).

17 Storage engines and indexing internals ADVANCED

▶ Source: "how databases scale writes", indexing and proximity-search videos

B+ tree vs log-structured writes

Relational DBs index with **B+ trees**: O(log N) search and insert, but heavy random disk I/O and per-write acknowledgements bottleneck under write spikes. The fastest possible write is an **append to a log** (O(1)), but reading an unindexed log is an O(N) scan. The **LSM tree** gets both.



LSM tree: writes go to a WAL and a sorted in-memory MemTable, flush as immutable SSTables, and are merged by compaction.

Piece	Role
WAL	Append-only log written first, for crash recovery
MemTable	Sorted buffer in RAM: batches writes, avoids disk seeks
SSTable	Immutable, **sorted** file flushed from the MemTable; enables O(log N) binary search
Compaction	Background **merge sort** of many small SSTables into larger ones; purges duplicates/deleted keys; keeps reads fast
Bloom filter	Per-SSTable probabilistic check: **false = definitely absent** (skip the disk read); true = maybe present (Ch 18)

TRADE-OFF

B+ tree: balanced, great reads, costlier writes (traditional SQL). LSM: very high write throughput, reads need compaction and filters (Cassandra, HBase, many time-series and key-value stores).

Geospatial indexing: "who is near me?"

B-trees are 1-D. Sorting by longitude alone puts street neighbours millions of rows apart; two separate indexes force huge merges. Two approaches: map 2-D to 1-D so a B-tree works, or build spatial trees.

Technique	How it works	Notes
Geohash	Recursively splits the map into grid cells, encodes lat/long as a Base32 string; **shared prefix = nearby**; B-tree prefix range scans	**Boundary problem:** points 1 m apart across a cell edge get different prefixes. Fix: query the cell + its **8 neighbours (3×3)** and post-filter by exact distance
Quadtree	Recursively splits space into 4 quadrants when a cell exceeds K points	Adapts to density; variable depth, pointer-heavy (random disk reads)
KD / BKD tree	Alternates split dimension per level; BKD packs nodes into disk-page blocks	Powers Elasticsearch geo queries
R-tree	Groups items in **Minimum Bounding Rectangles**; balanced, equal depth	Handles polygons (delivery zones, highways); PostGIS, SQLite Spatial. Preferred in production
Google S2	Earth projected on a cube; 64-bit hierarchical cell IDs; no pole distortion	MongoDB `2dsphere`
Uber H3	**Hexagonal** cells, 16 resolutions; all 6 neighbours equidistant	Dispatch, surge pricing, analytics

18 Specialised stores: DynamoDB, Elasticsearch, time series, probabilistic structures ADVANCED

▶ **Source: DynamoDB, Elasticsearch, time-series and big-data-structures videos**

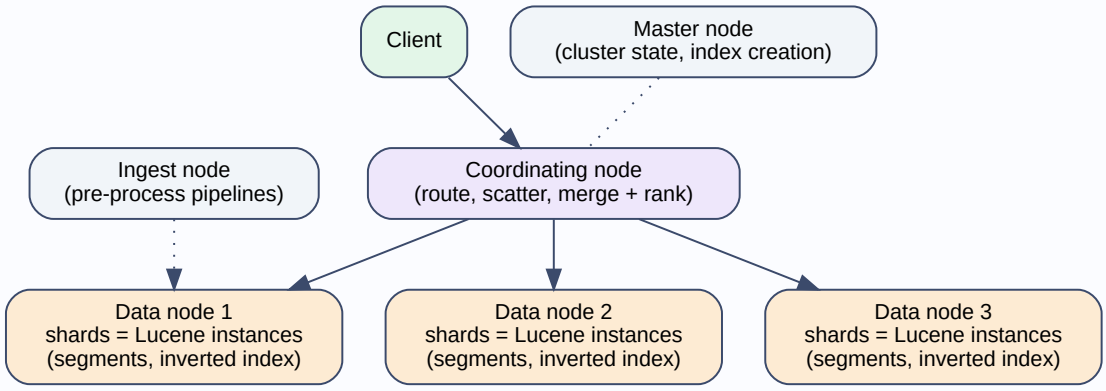
DynamoDB

Concept	Detail
Model	Table → items (rows) → attributes. **Schemaless**: items may differ; the app handles missing attributes
Partition key (PK)	Passed through consistent hashing to choose the physical partition
Sort key (SK)	Optional; ordered (B-tree) within a partition for range queries. **Composite key** PK+SK: `ChatID + MessageID` keeps a chat's messages together and chronological
GSI	New partition key (+ sort key) across the whole table for another access pattern (messages by `UserID`); asynchronously maintained
LSI	**Same** partition key, **different** sort key (sort a chat's messages by attachment count)
Replication	Single-leader, async across AZs; **sloppy quorum** lets writes proceed during partitions. Reads: **eventual** (default, any replica, sub-10 ms) or **strong** (leader)
Extras	ACID transactions up to 100 items · **DAX** (managed in-memory cache, microseconds) · **Streams** (CDC → Lambda / Elasticsearch)

Use: huge key-value / predictable access patterns, auto-scaling. **Avoid:** deep joins, transactions over >100 items, designs needing many GSIs/LSIs.

Elasticsearch

Search engine built on **Apache Lucene**. **Document** = JSON; **index** = collection of documents; **mapping** = schema (`text` tokenised for full-text, `keyword` exact match, `float` ranges). **Denormalise** (embed reviews inside books): no joins.



Node roles: coordinating (route/merge), master (cluster state), data (shards), ingest (pipelines).

- **Shards** are independent Lucene instances made of **immutable segments** (updates/deletes use soft-delete bitmasks): lock-free concurrent reads and OS page caching. **Inverted index** (term → doc IDs) powers search; **doc values** (columnar) power sort/aggregations.

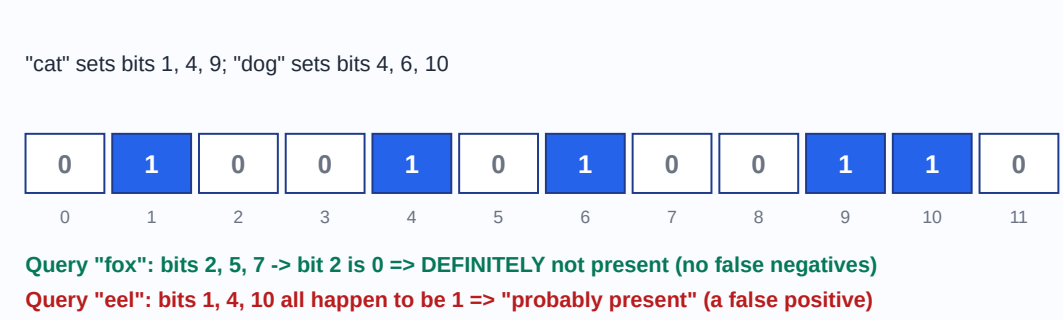
- **Relevance:** BM25 / TF-IDF (term frequency vs rarity).
- **Pagination:** `from + size` (slow when deep) → `search_after` (cursor-like, efficient) → **Point in Time** (consistent snapshot, no dupes/misses).

> **NEVER THE SOURCE OF TRUTH**
> Pair Elasticsearch with a durable primary DB (Postgres/DynamoDB) synced via **CDC / streams**. It shines for read-heavy search, geospatial and vector workloads.

Time-series databases (TSDB)

- **Why SQL fails:** metrics arrive at 50k–500k writes/s (SQL tops out near ~10k) and raw rows bloat; "average CPU over 8 hours" becomes a multi-minute scan of billions of rows.
- **Append-only + LSM:** WAL → MemTable → immutable segments → compaction; avoids random disk seeks.
- **Compression: delta encoding** (store differences); **delta-of-deltas** (Gorilla): regular 10 s timestamps give deltas 10,10,10 → delta-of-deltas 0,0,0 ≈ **1 bit per timestamp**.
- **Query speed:** partition by time and tags; block min/max metadata and Bloom filters skip files; **downsampling** rolls old 10 s data into 1 min / 1 h resolution.
- **Failure mode: high cardinality.** A series = metric + unique tag set; tags like `user_id` / `order_id` explode into millions of series, strain memory and indexes. Track those in a general DB instead.

Probabilistic data structures for big data



Bloom filter: k hash functions set bits. A 0 proves absence; all-1s only suggests presence.

Structure	Answers	How / use
Bloom filter	Is X in the set? **No** (certain) or **probably**	Bit array + K hashes; an order of magnitude less space than a hash set; **no false negatives**, tunable false positives. Web crawlers (visited URLs), checking a key before a DB/disk lookup. Can't store or list items
Count-Min Sketch	How often did X occur? (**upper bound**)	K hashes × 2-D counter array; increment on write, take the **min** across rows on read to damp collisions. Heavy hitters (viral posts, top API callers, abusive IPs); pair with a bounded priority queue to know *which* keys
HyperLogLog	How many **unique** items?	Rare long runs of trailing zeros in hashes imply ~2^N distinct items; many buckets + **harmonic mean** tame outliers. Fixed tiny memory for DAU, unique visitors, column cardinality (Redis `PFCOUNT`)

19 Redis in depth ADVANCED

▶ Source: L23

Redis is an in-memory key-value store (sub-millisecond, ~100× faster than disk DBs) that doubles as a message broker (Pub/Sub, Streams), distributed lock engine and geo index.

Why so fast, and what it costs

- **RAM only** (no disk I/O); a **single-threaded event loop** (no context switching, locks or races); optimised C structures (skip lists, ziplists).
- **Trade-offs:** volatile; persistence via **AOF** and **RDB snapshots** reduces but doesn't eliminate loss (AWS MemoryDB adds multi-AZ durable logs); **eventually consistent** as a cache (stale until TTL/invalidation).

Data structures

Type	Commands	Use
String (≤512 MB)	`SET GET INCR DECR` ; `NX` = only if absent	Cache values, counters, locks
Hash	`HSET HGET HGETALL`	Objects (profile, product) without JSON serialisation
List	`LPUSH RPUSH LPOP RPOP`	Queues, stacks, task buffers
Set / Sorted set	`SADD SREM` / `ZADD ZRANGE`	Unique members / leaderboards, priority queues, sliding-window limiters
Geo	`GEOADD GEOSEARCH`	Nearby drivers/stations within a radius
Pub/Sub, Streams		Fire-and-forget broadcast / Kafka-like append log

Deployment architectures

Setup	How	Limit
Single node	One instance	SPOF, no failover
Master-replica	Master takes writes, replicas serve reads, async replication	Reads scale, **writes don't**
Cluster	Keys sharded over masters using **16,384 hash slots**: `slot = hash(key) % 16384` . Nodes share topology by **gossip**; clients cache the slot map and route directly; a wrong node answers `MOVED` so the client refreshes	Multi-key ops / transactions across slots restricted

Production patterns

Pattern	Design
Hot key (one celebrity key gets millions of req/s on one node)	(1) more read replicas; (2) **key salting**: store N copies `user:42_1..N` so hashing spreads them; (3) **local in-app cache** (e.g. 100 ms TTL LRU) to skip the network

Rate limiter	`INCR` a per-user/IP counter with `EXPIRE` = window (600 s); over threshold → `429` until TTL clears it
Distributed lock (BookMyShow seat hold)	`SET seat:12A user NX EX 600` : succeeds only for the first caller; TTL auto-releases if the holder crashes, preventing double booking
Session store	`sess:<id>` with TTL (Ch 11)

> **BENCHMARK TO QUOTE**
> 1M-row product table: direct PostgreSQL query ≈ 117–126 ms vs cached JSON from Redis ≈ 7–10 ms (>10× faster).

20 Real-time at scale: WebSockets and WebRTC

ADVANCED

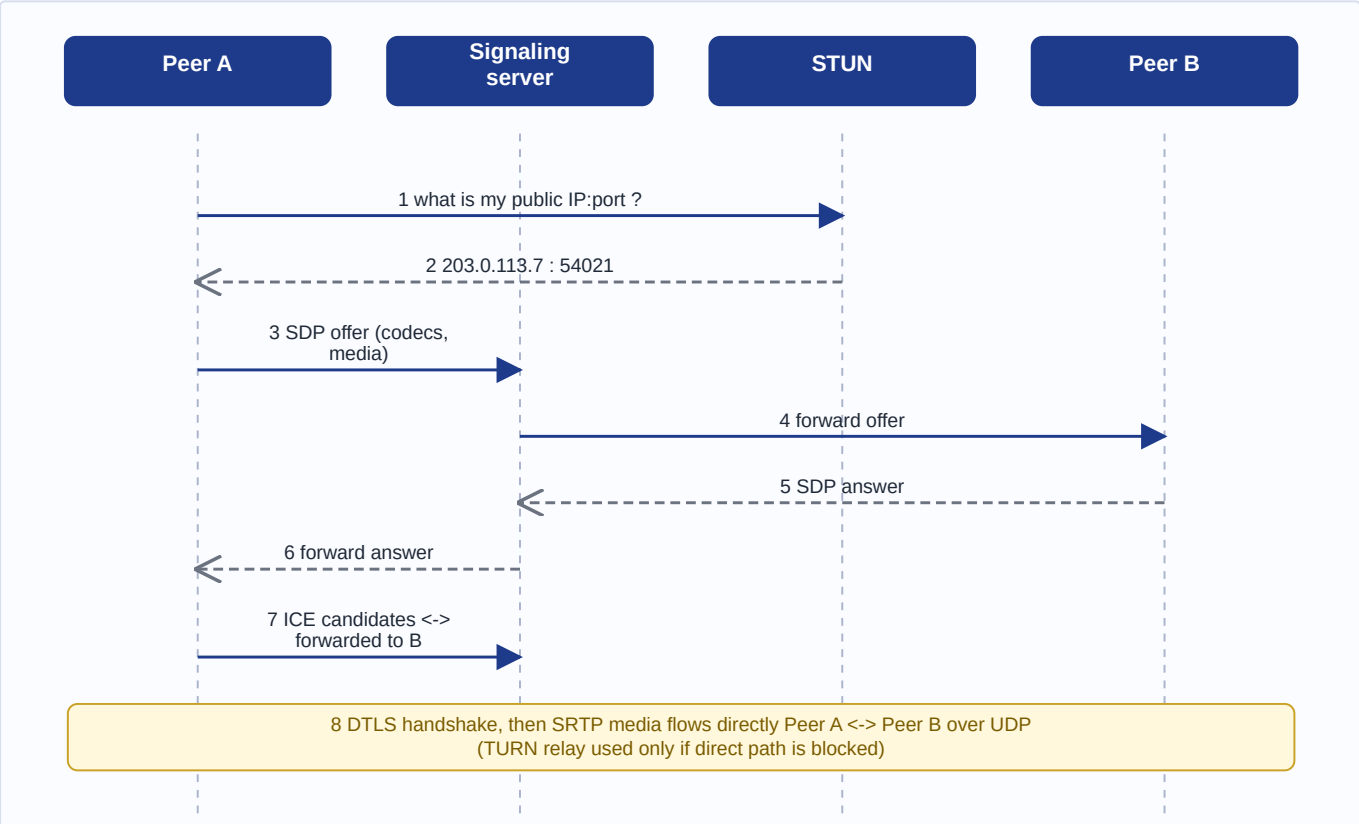
▶ Source: L28, L31

Scaling WebSockets

- **Stateful:** each connection is pinned to one server, so the LB needs **sticky routing** (L4 / IP-hash) and servers can't be treated as interchangeable.
- **Cross-server delivery:** User A is on server 1, User B on server 2, so server 1 can't push to B. Use a **Pub/Sub broker (Redis Pub/Sub)**: server 1 publishes, server 2 (subscribed) pushes to B over its socket. Keep a **connection registry** (user → server).
- **Resilience:** ping/pong heartbeats, client reconnect with exponential backoff, app-level message IDs + ACKs.

WebRTC: video calls like Zoom

WebRTC is a browser-native framework for secure, low-latency peer-to-peer audio, video and data, with no plugins. Obstacle: two browsers behind different **NATs** don't know each other's public IP/port or firewall rules.

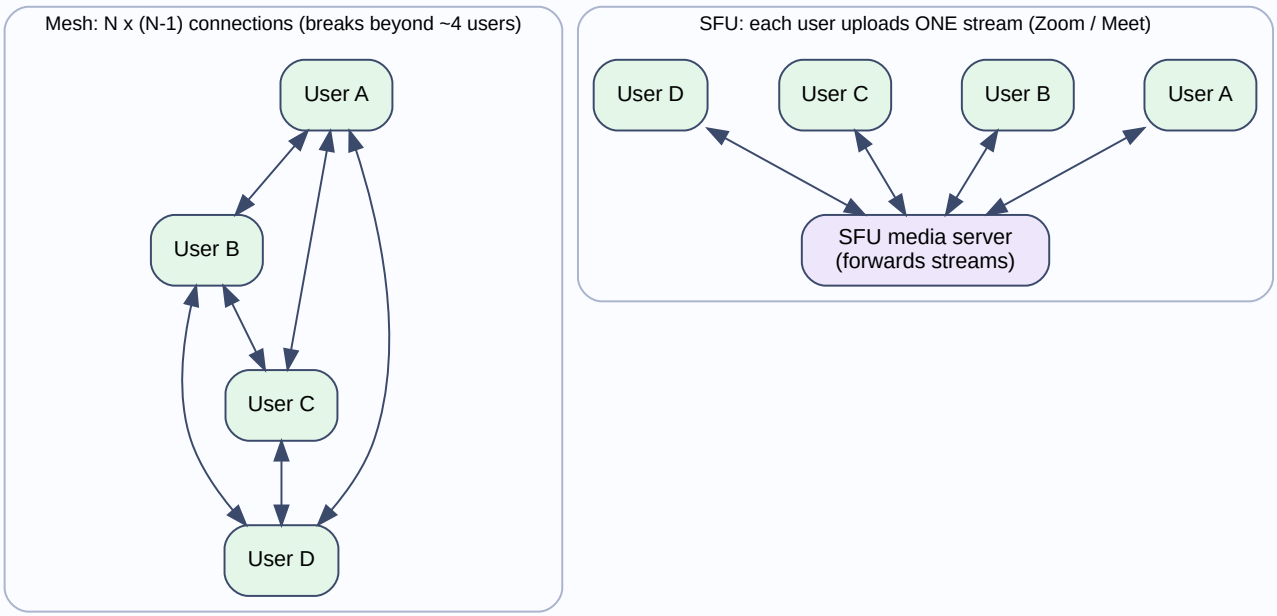


Signaling sets up the call; STUN discovers public addresses; media then flows peer-to-peer over UDP.

Piece	Role
Signaling server	Your backend; lets peers find each other and exchange **SDP** offer/answer (codecs, media types) and ICE candidates. It carries **no media**
ICE candidate	A possible network path: **host** (local IP, same LAN), **server-reflexive** (public IP via STUN), **relay** (via TURN). Tried in that order; first working path wins

STUN	Tells a client its public IP:port. Lightweight (free public servers are fine)
TURN	Fallback **relay** when symmetric NAT / strict firewalls block direct P2P. Always works but costs bandwidth and latency
DTLS + SRTP	DTLS handshake exchanges keys; SRTP encrypts media
UDP transport	Speed over perfection: occasional lost frames beat TCP retransmission delay. WebRTC handles adaptive bitrate, loss concealment and congestion control

Group calls: mesh vs SFU



Mesh needs N×(N−1) connections and collapses client CPU/bandwidth; an SFU makes each user upload once.

SFU (Selective Forwarding Unit), used by Zoom and Google Meet: each participant encodes and uploads **one** stream; the SFU forwards it to everyone else. Client uploads drop from N−1 to 1, so calls scale to hundreds. Host the SFU yourself (core infrastructure); signaling lives in your backend; TURN can be self-hosted or managed.

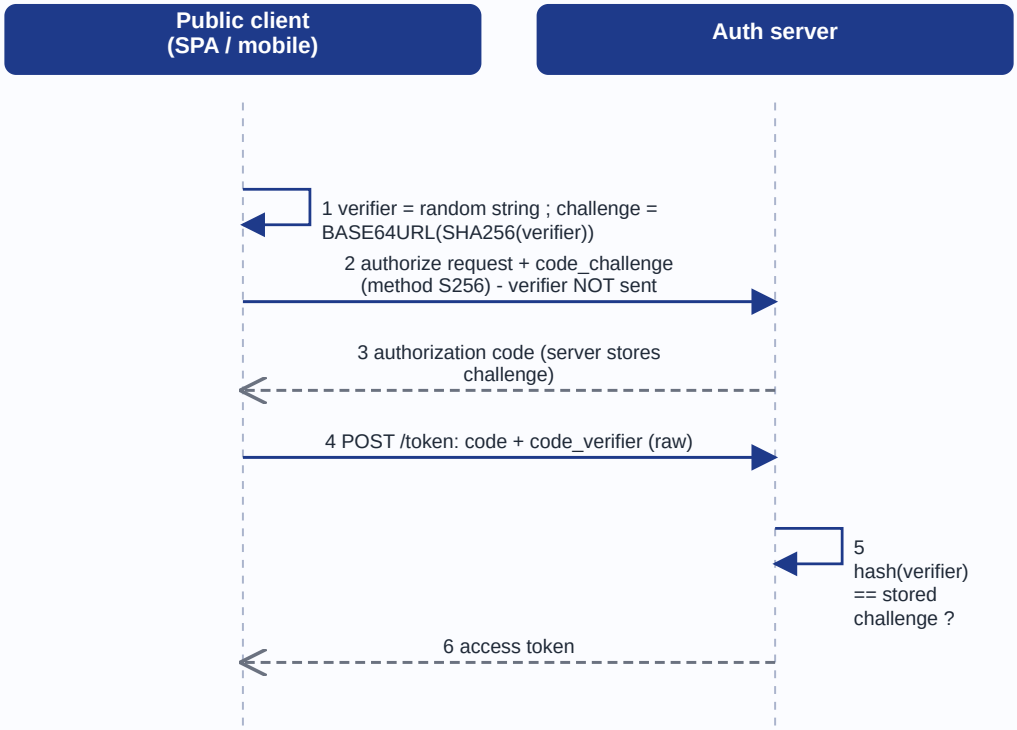
21 Advanced security and identity ADVANCED

▶ Source: OAuth PKCE, L33 (JWT), L29, L10

PKCE: OAuth for public clients

Confidential clients (server backends) can keep a `client_secret`. **Public clients** (React/Angular SPAs, Android/iOS/Flutter apps) cannot: code can be decompiled, so they get only a client_id. Then a stolen **authorization code** (via a hijacked custom URL scheme or link) could be exchanged for tokens by an attacker.

PKCE ("pixy", Proof Key for Code Exchange) replaces the static secret with a **per-request proof**: `code_verifier` = random high-entropy string kept in memory; `code_challenge` = `BASE64URL(SHA256(verifier))` (one-way: the challenge reveals nothing about the verifier).



The challenge goes out in the front channel; the raw verifier is shown only at token exchange.

	Classic (confidential)	PKCE (public)
Credential	Static `client_secret`	Dynamic verifier/challenge pair
Authorize request adds	(nothing extra)	`code_challenge`, `code_challenge_method=S256`
Token request proof	`code` + `client_secret`	`code` + `code_verifier`

An attacker holding only the stolen code can't produce the verifier, so the server issues nothing. Modern guidance recommends PKCE for all clients.

JWT in production

Topic	Detail
JWS vs JWE	**JWS** (default): signed, readable, tamper-proof. **JWE**: encrypted payload, only if sensitive data must travel in it
HS256 vs RS256	**HS256**: one shared secret signs *and* verifies; every service must hold it. **RS256**: private key (auth server only) signs, public key verifies. **Prefer RS256 for distributed systems**
Algorithm confusion	Never trust the token's `alg` header (attackers switch to `none` or HS256). Enforce the expected algorithm in code
Refresh-token rotation	Every refresh issues a **new refresh token and invalidates the old**. **Reuse detection:** if an old one reappears, assume theft and revoke all the user's tokens. Store refresh tokens server-side (Redis/DB) in `HttpOnly Secure` cookies
Secrets	Keys from Secrets Manager/Vault are loaded into memory at boot, so verification stays a local in-memory operation
Gateway verification	Verify at the API gateway once at the edge; auth server isn't a SPOF for existing sessions

mTLS and zero trust

Standard TLS authenticates only the server. **Mutual TLS**: both sides present certificates and verify each other. Ideal for **service-to-service** traffic and high-security banking APIs where shared API keys are undesirable.

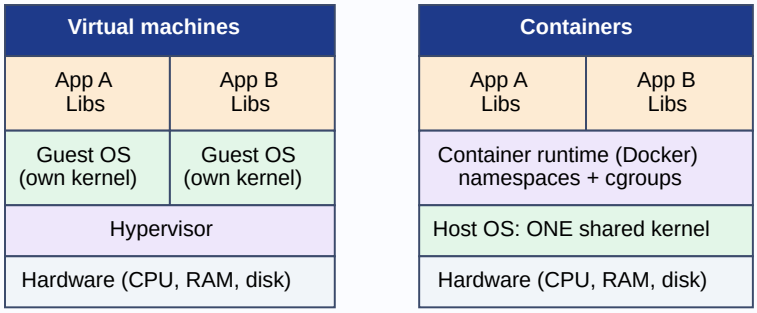
gRPC and streaming notes

- gRPC streaming (binary Protobuf on HTTP/2) beats GraphQL subscriptions (text JSON over WebSockets) on latency and overhead.
- Browsers lack native gRPC (use gRPC-Web proxies), which is why WebSockets stay popular for browser-facing real-time features.

22 Infrastructure and theory odds and ends ADVANCED

▶ **Source: VM vs container, Lamport clock videos, L24**

Virtual machine vs container



VMs virtualise hardware (each with its own kernel); containers virtualise the OS (one shared kernel).

	VM	Docker container
Isolation	Hypervisor divides CPU/RAM/disk; each VM boots a **full guest OS with its own kernel**	An isolated **process** on the host; **shared host kernel**
Mechanism	Hardware-level virtualisation	Linux **namespaces** (what a process can see) + **cgroups** (CPU/RAM limits)
Start / size	Minutes; image ~600 MB (Ubuntu)	Seconds; ~30 MB
Security	Strong isolation	Weaker: a kernel vulnerability could allow breakout

Linux containers need a Linux kernel; Docker Desktop on macOS silently runs a Linux VM.

Lamport logical clock

Problem: physical clocks on servers in different regions can't be perfectly synchronised, so timestamps can't reliably order events across machines. **Solution:** a per-process integer counter that tracks **causal order**:

- Internal event or send: `counter = counter + 1` (send the counter with the message).
- Receive with timestamp T: `counter = max(local, T) + 1`.

PACELC: CAP for normal operation

CAP only describes behaviour *during a partition* (rare). **PACELC:** if **P**artition, choose **A** or **C**; **E**lse (normal), choose **L**atency or **C**onsistency.

Database	Class	Meaning
DynamoDB	PA / EL	Available in a partition; low latency over consistency normally (eventual reads)
Cassandra	PA / EL	Same: async background replication for speed
Google Spanner	PC / EC	Consistent in a partition; strong consistency globally, accepting sync latency

PART 3

Part 3 · Supplement

Added material, not from your lecture notes. Standard system-design topics that interviews often probe and the lectures did not cover. Kept to the crux: definition, mechanism, and what to say.

23 Supplement: reliability, caching failures, IDs and concurrency FOUNDATIONAL ADDED

1 · Delivery semantics (queues, Kafka, webhooks)

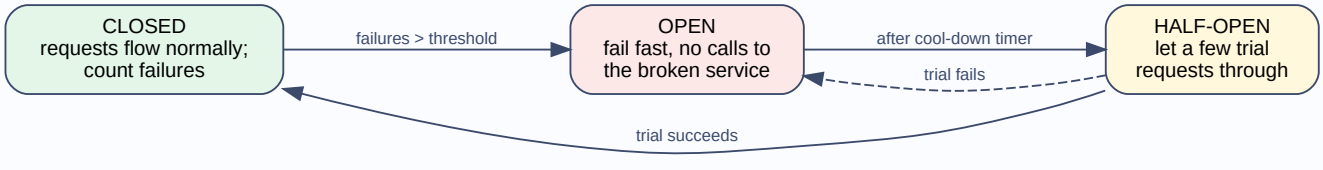
Guarantee	Meaning	Reality
At-most-once	Sent once, never retried; may be lost	Fine for metrics, logs, location pings
At-least-once	Retried until ACKed; never lost, **may duplicate**	**The practical default** (SQS, RabbitMQ, Kafka)
Exactly-once	Processed once, no loss, no duplicates	Not truly achievable across a network. Build it as **at-least-once + idempotent consumer** (dedupe by message/event ID). Kafka offers idempotent producers and transactions within its own ecosystem

> **SAY THIS**
> "I'll use at-least-once delivery and make consumers idempotent, because exactly-once end to end doesn't exist across independent systems."

2 · Resilience patterns

In a distributed system something is always failing. One slow dependency can exhaust threads upstream and cause a **cascading failure**. Defences:

Pattern	What it does
Timeouts	Never wait forever; set a deadline on every network call
Retries + exponential backoff + jitter	Wait 1 s, 2 s, 4 s… plus random jitter so clients don't retry in sync (retry storm). Retry only **idempotent** operations, with a cap
Circuit breaker	After repeated failures stop calling the dependency and fail fast; probe it periodically (diagram below)
Bulkhead	Separate thread/connection pools per dependency so one slow service can't starve the rest
Load shedding / backpressure	Under overload, reject low-priority requests (429/503) instead of collapsing; bounded queues push back on producers
Graceful degradation	Serve a fallback (cached or default recommendations) when a non-critical service is down



Circuit breaker state machine.

3 · Cache failure modes

Problem	What happens	Fix
Stampede (thundering herd)	A hot key expires; thousands of requests miss at once and hit the DB	Lock / single-flight so one request rebuilds it; refresh early; **jittered TTLs**
Penetration	Requests for keys that **don't exist** always miss and hit the DB (also an attack)	Cache the null result briefly; **Bloom filter** of valid keys; validate input
Avalanche	Many keys expire together, or the cache node dies, so the DB is flooded	Randomise TTLs; replicas / cluster; rate-limit and warm the cache

Redis vs Memcached: Redis has rich data structures, persistence, replication and clustering; Memcached is a simple, multithreaded string cache. Default to Redis.

4 · Availability and performance metrics

Availability	Downtime / year	Note
99.9% ("three nines")	~8.8 hours	Typical single-region service
99.99%	~53 minutes	Needs multi-AZ redundancy
99.999%	~5 minutes	Very costly; multi-region

- **SLI** = the measured indicator (e.g. % successful requests). **SLO** = internal target (99.9%). **SLA** = contract with penalties, looser than the SLO.
- **Components in series multiply** availability: two 99.9% services in a chain give ≈ 99.8%. **Redundancy in parallel** raises it.
- **Latency percentiles:** averages hide pain. p50 = median; **p99** = the slowest 1% (tail latency). At scale the tail hits many users, so quote p95/p99.
- **Latency** = time per request; **throughput** = requests per second. Batching raises throughput but can raise latency.

5 · Disaster recovery

Term	Meaning
RPO	Recovery Point Objective: how much data you can afford to lose (time since last good copy)
RTO	Recovery Time Objective: how long you can be down

Strategy	RTO / RPO and cost
Backup and restore	Hours; cheapest
Pilot light / warm standby	Minutes; a scaled-down copy runs in another region
Active-passive multi-region	Standby region takes over on failure
Active-active multi-region	Near-zero; most expensive and hardest (write conflicts, replication lag, CAP trade-offs)

Also: automated, **tested** restores, point-in-time recovery, and cross-region replication of backups.

6 · Database concurrency

Isolation level	Prevents
Read uncommitted	Nothing (dirty reads possible)
Read committed	Dirty reads (default in PostgreSQL)
Repeatable read	+ non-repeatable reads (default in MySQL InnoDB)
Serializable	+ phantom reads; behaves as if transactions ran one by one; slowest

Dirty read = seeing uncommitted data · non-repeatable = a row changes between two reads · phantom = new rows appear between two range queries.

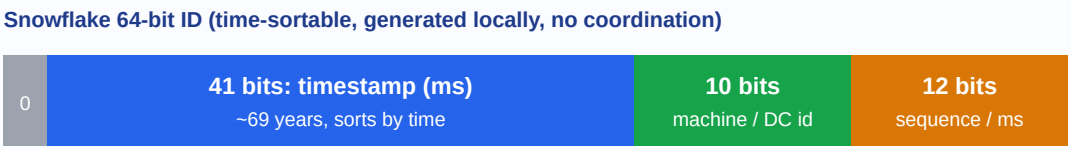
	Pessimistic locking	Optimistic locking
Idea	Lock first (`SELECT … FOR UPDATE`)	No lock; check a **version column** on write (`UPDATE … WHERE id=1 AND version=7`); 0 rows updated = conflict, retry
Best when	High contention (ticket seats, inventory)	Low contention, read-heavy

Normalisation removes duplication (clean writes, joins on read); **denormalisation** duplicates data to make reads fast (NoSQL, read models), at the cost of keeping copies in sync.

7 · Building blocks

Unique ID generation

Auto-increment breaks across shards (collisions, single bottleneck). **UUID v4**: 128-bit random, no coordination, but large, unordered and index-unfriendly. **Snowflake-style IDs** are the standard answer: 64-bit, **time-sortable**, generated locally.



Up to 4,096 IDs per millisecond per machine; two machines never collide because the machine id differs.

Snowflake layout: timestamp | machine id | sequence.

Forward vs reverse proxy

	Forward proxy	Reverse proxy
Sits in front of	Clients (hides the client)	Servers (hides the servers)
Examples	Corporate proxy, VPN	NGINX, load balancer, API gateway, CDN edge

Leader election and heartbeats

Systems needing one leader (primary DB, Kafka partition) use **heartbeats**; if the leader goes silent, nodes elect a new one by **majority vote** (Raft, ZooKeeper, etcd), which prevents split brain (Ch 15).

24 Supplement: observability, deployment and data pipelines ADVANCED ADDED

Observability: knowing what the system is doing

Pillar	What	Tools
Logs	Discrete events (errors, requests); structured JSON, centralised	ELK / OpenSearch, Loki
Metrics	Numeric time series (QPS, p99, CPU)	Prometheus, Grafana, CloudWatch
Traces	One request followed across services via a **correlation / trace ID**	OpenTelemetry, Jaeger, Zipkin

Four golden signals: latency, traffic, errors, saturation. Alert on **SLO symptoms** (error rate, p99), not every CPU blip.

Deployment and infrastructure

- **Service discovery:** instances come and go, so IPs change. A registry (Consul, Eureka, etcd) or Kubernetes DNS maps service names to live instances; **client-side** (caller picks from the list) or **server-side** (via LB), as in Ch 3.
- **Kubernetes basics:** a **pod** runs one or more containers; a **Deployment** keeps N replicas running and rolls updates; a **Service** gives pods a stable address; the **HPA** autoscales on CPU or custom metrics; failed pods are rescheduled automatically (self-healing).

Release strategy	How	Trade-off
Rolling	Replace instances gradually	Mixed versions run briefly
Blue-green	Full new environment; flip traffic at the LB; flip back to roll back	Double capacity during the switch
Canary	Send 1–5% of traffic to the new version, watch metrics, then ramp up	Needs good monitoring
Feature flags	Ship code dark, enable per user/percentage at runtime	Flag debt to clean up

Data pipelines and analytics

	OLTP	OLAP
Purpose	Live transactions (orders, payments)	Analytics over huge history (revenue by region)
Store	PostgreSQL, MySQL, DynamoDB	Data warehouse (BigQuery, Snowflake, Redshift): columnar

- **Data lake:** cheap raw storage (S3); **warehouse:** cleaned, fast SQL; **lakehouse:** both in one.
- **Batch** (Spark, scheduled): process large data periodically, high throughput, minutes–hours latency. **Stream** (Flink, Kafka Streams): process events continuously, seconds latency (fraud detection, live dashboards).
- **ETL** transforms before loading; **ELT** loads first, transforms in the warehouse (modern default). DBs feed pipelines via **CDC**.

PART 4

Part 4 · Interview Playbook

A quick-reference map from requirement to component, and a concept-level checklist for structuring any HLD answer.

25 Which component for which problem QUICK REFERENCE

Requirement → component

If the problem says…	Reach for	Chapter
Traffic growing, one server overloaded	Horizontal scaling + load balancer	3
No downtime; component can't be a SPOF	Redundancy, health checks, active-passive/active-active, multi-AZ	3
Read-heavy, repeated data, slow queries	Cache (Redis) + TTL; CDN for static/media	4, 19
Core transactional data (orders, payments)	PostgreSQL / MySQL (ACID)	5
Flexible schema / massive writes / logs	Document store / Cassandra / DynamoDB	5, 18
Reads far exceed writes	Read replicas (+ cache)	5, 15
Writes or storage exceed one machine	Sharding + consistent hashing	14
Images, video, files	Object storage + presigned URLs + CDN; metadata in DB	5
Full-text search, relevance	Elasticsearch (fed by CDC)	18
"Nearby" drivers/restaurants	Geohash / quadtree / R-tree / H3; Redis GEO	17, 19
Metrics, telemetry, sensor data	Time-series DB (LSM + compression + downsampling)	18
Unique counts / frequency / membership at huge scale	HyperLogLog / Count-Min Sketch / Bloom filter	18
Many clients, many services, one front door	API gateway (auth, routing, rate limit, aggregation)	10
Team/service independence, independent scaling	Microservices (DB per service)	10
Public/simple CRUD API	REST (+ cursor pagination for feeds)	6
Mobile needs flexible nested data	GraphQL	7
Fast internal service calls / streaming	gRPC	7
Chat, live updates, games, collaborative editing	WebSockets (+ Redis Pub/Sub to scale)	8, 20
Video/voice calls	WebRTC + signaling + STUN/TURN + SFU	20

3rd-party notifies you (payments, CI)	Webhook: verify, enqueue, 200 fast, idempotent	8
Heavy/slow work, spikes, decoupling	Message queue + workers + DLQ	9, 16
One event, many reactions; replay; huge throughput	Kafka / pub-sub fan-out	9, 16
Multi-service business transaction	Saga + transactional outbox	15
Audit trail; read and write loads differ	Event sourcing; CQRS	15
Login, API access, 3rd-party login	Sessions (Redis) / JWT + refresh / OAuth 2.0 (PKCE for SPA/mobile)	11, 21
Abuse, brute force, DDoS	Rate limiting (token bucket) at the gateway	11
Prevent double booking / oversell	CP choice + Redis lock `SET NX EX` / DB transaction	12, 19
Dependency slow/failing; avoid cascading failure	Timeouts, backoff + jitter, circuit breaker, bulkhead, load shedding	23
Queue consumer may see duplicates	At-least-once + idempotent consumer	23
Hot key expires / non-existent keys hammer DB	Stampede lock, TTL jitter, null caching, Bloom filter	23
"Four nines", SLA, tail latency	Nines table, SLO, p99, redundancy in parallel	23
Region outage, data loss limits	RTO/RPO, warm standby, multi-region	23
Concurrent updates / double booking in the DB	Isolation levels, optimistic vs pessimistic locking	23
Globally unique, sortable IDs across shards	Snowflake IDs	23
Debugging and monitoring a distributed system	Logs + metrics + traces, golden signals	24
Safe releases, scheduling containers	Canary / blue-green, Kubernetes, service discovery	24
Analytics over history, real-time processing	Warehouse/lake, batch vs stream, CDC	24
Staleness acceptable, uptime critical	AP / eventual consistency	12

The default stack (when unsure)

SAFE DEFAULTS TO STATE ALOUD

TCP/HTTPS + REST + JSON · stateless app servers behind a **load balancer** · **PostgreSQL** primary with read replicas · **Redis** cache-aside with TTL · **CDN + S3** for media · **queue/Kafka** for async work · **JWT or session in Redis** · **API gateway** with rate limiting. Deviate only when a requirement justifies it.

Single-point-of-failure audit

Component	Remove the SPOF with
App server	N stateless instances + LB + auto-scaling
Load balancer / gateway	Active-passive or active-active; managed multi-AZ
Database	Primary + replicas with automatic failover; sharding for write scale
Cache	Master-replica or cluster; remember the DB must survive a cold cache
Message broker	Clustering / quorum queues / replicated Kafka partitions
Auth server	Stateless JWT verification keeps existing traffic alive

A concept-level checklist for any HLD answer

Worked walkthroughs (WhatsApp, Uber, …) come in the next module; this is the skeleton they all follow.

1. **Clarify requirements:** functional, plus non-functional (scale, latency, availability, consistency needs).
2. **Estimate:** users → requests/second, storage/day, bandwidth, cache size (Ch 13).
3. **Define the API:** REST/gRPC/GraphQL, key endpoints, pagination, idempotency, auth (Ch 6, 7, 11).
4. **Choose data stores:** default Postgres; add NoSQL/search/object storage per access pattern (Ch 5, 18).
5. **Draw the reference architecture:** client → CDN/DNS → LB → gateway → services → cache → DB (see the diagram at the start).
6. **Scale the hot paths:** caching, replicas, sharding with a good shard key, async queues (Ch 4, 14, 16).
7. **Decide consistency:** CP vs AP per feature; eventual consistency where acceptable (Ch 12).
8. **Harden:** remove SPOFs, retries with backoff + idempotency, DLQs, rate limits, security (Ch 11, 23).
9. **State trade-offs:** every choice costs something; say what and why.

THE MINDSET

Start simple, find the bottleneck, apply the lightest component that fixes it, and always name the trade-off (consistency vs latency, simplicity vs scale, cost vs durability).